

NILO - Modelo de Processo para o desenvolvimento de linguagens de programação

1 INTRODUÇÃO

Iniciar o aprendizado na área da computação é uma tarefa árdua, pois os fundamentos não são de fácil compreensão pela grande quantidade de conceitos acoplados (Arimoto; Oliveira, 2019). Além disso, o curso de Tecnologia em Análise e desenvolvimento de sistemas (TADS) do Instituto Federal do Rio Grande Norte (IFRN) do campus Natal Central (CNAT) possui uma matriz curricular definida pelo projeto pedagógico do curso (PPC) em que o enfoque de ensino é o desenvolvimento de sistemas de alto nível de abstração por meio de ferramentas e linguagens que contribuem e simplificam as implementações (Instituto Federal De Educação, Ciência E Tecnologia Do Rio Grande Do Norte, 2012).

Consequentemente, existe uma carência no ensino de alguns segmentos da computação no curso TADS, como o de criação de linguagens de programação, que são sistemas formais muito importantes a serem estudados na área da ciência da computação. Conhecer fundamentos para a criação de linguagens de programação pode permitir uma melhor articulação para manifestações de ideias, maior aptidão para aprender novas linguagens e crescimento do conhecimento em diferentes áreas da computação (Sebesta, 2011, p.20). Nesse sentido, a falta de aprofundamento na área de linguagens configura uma lacuna na formação dos estudantes do curso.

Nesse cenário, viu-se a necessidade de desenvolvimento de um modelo de processo para auxiliar os estudantes na sua jornada de aprendizado sobre o desenvolvimento de linguagens de programação e aprofundamento nos seus principais conceitos. Entretanto, esse modelo não indica de maneira determinística “o que deve ser feito”, mas oferece um arcabouço conceitual e prático que orienta o desenvolvimento. A responsabilidade pelo aprofundamento dos conteúdos e pela seleção das ferramentas adequadas é individual. Em síntese, este modelo de processo configura-se como um instrumento de apoio acadêmico, teórico e prático, que abre possibilidades metodológicas sem restringir a criatividade ou impor um fluxo único de construção de linguagens de programação.

2 ASPECTOS ESTRUTURAIS E UTILITÁRIOS DE LINGUAGENS

Uma linguagem de programação é uma formalização de instruções de alto nível usadas para gerar algoritmos para resolver problemas reais em diferentes domínios (Sebesta, 2011, p.23). Sua aplicabilidade abrange diferentes propósitos, desde a criação de *softwares*, até mesmo para controle de robôs, por exemplo. Nessa seção

iremos compreender diferentes termos que cercam o desenvolvimento e o uso diário de linguagens de codificação, com o objetivo de facilitar a compreensão de termos técnicos usados ao longo do modelo.

2.1 TRADUÇÃO DE CÓDIGO

Para que um algoritmo descrito em uma linguagem de programação seja compreendido por um computador é necessário que esse código passe por um processo de tradução para linguagem de máquina (AHO et al., 2007, p.01).

Com o desenvolvimento de linguagens de codificação com especificidades distintas, surgiram diferentes formas de tradução do texto-fonte para binários de máquina. Isso ocorreu com o objetivo de suprir as necessidades particulares de cada linguagem. Dentre essas formas de tradução podemos citar a compilação e a interpretação. A seguir, iremos aprofundar cada uma delas com o propósito de compreender melhor como uma máquina entende um algoritmo desenvolvido por um programador.

2.1.1 Interpretação

O primeiro fluxo de tradução que podemos pontuar é a interpretação, nela as linhas de código são lidas de instrução por instrução sem a geração de código intermediário nem executáveis (Freecodecamp, 2020).

Figura 01 - Fluxo de uma tradução utilizando um interpretador



Fonte: A autora adaptado de Aho et al.(2007, p.02).

A Figura 01 demonstra o fluxo de um interpretador, em que cada linha do código-fonte é processada sequencialmente, sendo possível, dependendo do texto-fonte fornecido pelo programador, informar valores de entrada para serem processados e/ou gerar valores de saída ao longo da execução do código (AHO et al., 2077, p.02).

Tal abordagem traz uma diminuição significativa nos passos que envolvem a tradução do programa e facilita o processo de depuração (Sebesta, 2011, p.48), todavia, essa arquitetura também apresenta limitações relevantes. Entre as principais desvantagens, destacam-se a diminuição da velocidade de execução e do aumento de memória necessária (Sebesta, 2011, p.49).

Atualmente, linguagens como python (Python, 2026) e ruby (Ruby, 2026) utilizam interpretadores para suas traduções de código-fonte para código de máquina. Essa escolha privilegia a flexibilidade e a facilidade de experimentação, ainda que em detrimento da eficiência de desempenho e da robustez.

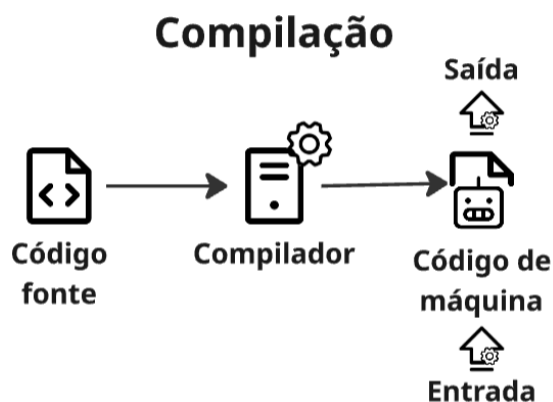
2.1.2 Compilação

No que diz respeito ao processo de compilação, o código-fonte é transformado em um arquivo executável que contém binários que podem ser executados diretamente pelo *hardware* (Aho et al., 2007, p.01).

Na compilação, podem ser identificados dois tipos principais. A compilação estática consiste na tradução completa do código-fonte para código de máquina antes da execução. Já a compilação dinâmica envolve a resolução de partes do código durante a execução, como o carregamento de bibliotecas externas ou a compilação em tempo de execução colaborando com a otimização do código (Cruz et al., 2008).

Além disso, apesar de uma mudança no fluxo de tradução, a compilação partilha de passos em comum com a interpretação. Assim, a distinção entre compilação e interpretação não reside em como o código é analisado e validado, mas sim na saída produzida que no caso do compilador é um executável e do interpretador é a execução imediata do código (Lee, 2017).

Figura 02 - Fluxo de uma tradução utilizando um compilador



Fonte: A autora adaptado de Aho et al.(2007, p.01).

A interação do programador após a compilação é apenas com o código executável (Lee, 2017), como representado na Figura 02. Isso, consequentemente, aumenta a velocidade de execução do código-fonte e otimiza o desempenho. Uma desvantagem que pode ser apontada na compilação está na complexidade de sua implementação, sendo portanto recomendada para projetos que exigem mais confiabilidade e segurança na implementação.

Por fim, é essencial uma boa compreensão das limitações e recomendações do uso de cada método de tradução para que assim seja escolhida a opção mais adequada para cada projeto de linguagem. A seguir iremos observar no Quadro 01 um panorama geral sobre cada forma de tradução vista neste subtópico.

Quadro 01 - Comparação entre métodos de tradução

Método de Tradução	Recomendado para	Limitações	Exemplos reais de utilização
Compilação	- Projetos que exigem um grande desempenho e execução direta no hardware ou em caso que queira um aprendizado mais aprofundado de fundamentos de linguagens de programação.	- As traduções são mais longas. - Possui menos flexibilidade para alterações em tempo de execução.	C, C++, Rust, Pascal, etc.
Interpretação	Em casos que	- Menor	Python, Ruby,

	necessitem de flexibilidade, testes rápidos e depuração fácil.	desempenho. - Menor segurança.	PHP, etc.
--	--	-----------------------------------	-----------

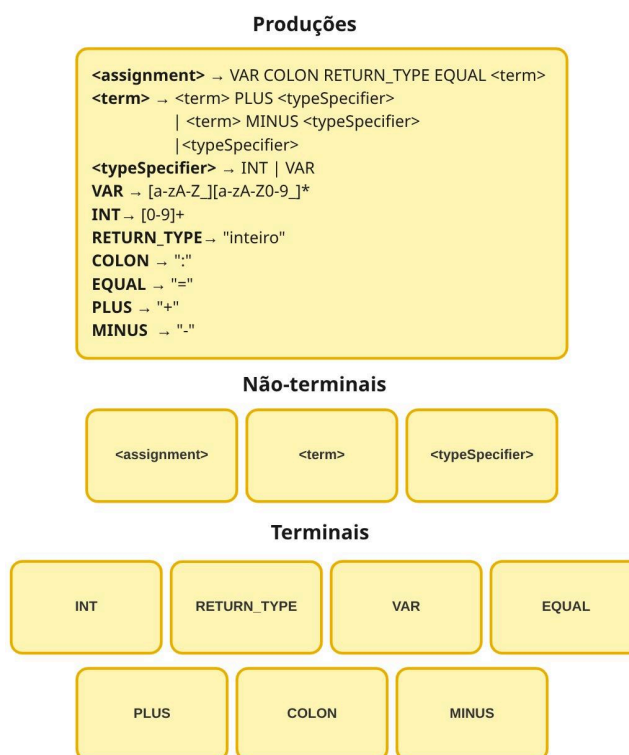
Fonte: A autora.

2.2 GRAMÁTICAS

Todo idioma possui uma gramática que dita as suas regras na forma escrita e falada, para que assim seja possível a formação de sentenças com aceção (Neves, 2002). O contexto de línguas faladas pelos indivíduos não se difere muito ao de linguagens de programação. Essa semelhança se explica pelo fato que a criação de linguagens de codificação necessita da definição de uma gramática como método formal de representação da sintaxe (Sebesta, 2011, p.139).

Além disso, existem diferentes tipos de gramáticas que se adaptam às necessidades do projeto de linguagem em desenvolvimento. O linguista norte-americano Chomsky (1959) desenvolveu uma categorização das gramáticas e linguagens que se denomina hierarquia de Chomsky. A categorização foi dividida em 4 classes que são gramáticas: irrestritas, sensíveis ao contexto, livres de contexto e regulares. Essas classes foram baseadas na complexidade e na capacidade de a partir de um conjunto finito de regras a linguagem poder se desdobrar em infinitas sentenças aceitáveis. Porém, é importante ressaltar que essas classes foram criadas para a área da linguística e não de forma específica para a computação. Por isso, iremos focar apenas na mais utilizada na área da computação que são as gramáticas livres de contexto (GLC)(Might, [20--]).

Figura 03 - Recorte de uma gramática livre de contexto



Fonte: A autora.

As gramáticas livres de contexto são definidas como gramáticas cujas regras, chamadas de produções, possuem um único símbolo não-terminal no lado esquerdo. Em outras palavras, cada produção segue o formato $A \rightarrow \underline{V}$, em que A é um não-terminal e $\underline{V}$ é uma sequência de terminais e/ou não-terminais (Johann, 2020). Para compreender melhor essa definição, é necessário detalhar os elementos que compõem uma gramática:

- **Símbolos terminais** representam valores concretos e finais que não podem ser separados em partes menores.
- **Símbolos não-terminais** representam uma estrutura da linguagem que pode ser substituída por uma sequência de símbolos terminais e/ou não-terminais descritos no lado direito da produção.
- **Produções** no contexto de gramáticas de linguagens de programação são regras que representam como estruturalmente é formada uma funcionalidade da linguagem. De forma técnica as produções são descritas com um não-terminal do lado esquerdo um símbolo de “ $\rightarrow$ ” e uma sequência de símbolos do lado direito, que podem substituir o não-terminal.

A Figura 03 apresenta um recorte de uma gramática livre de contexto que representa uma produção de atribuição. Nela, um não-terminal denominado *<assignment>* é composto pelos terminais *VAR*, *COLON*, *RETURN_TYPE* e *EQUAL*,

seguidos pelo não-terminal *<term>*, responsável por representar a expressão atribuída à variável. O não-terminal *<term>*, por sua vez, é definido recursivamente, permitindo a formação de expressões compostas por operações de soma e subtração entre identificadores (*VAR*) e valores inteiros (*INT*), representados pelo não-terminal *<typeSpecifier>*. Dessa forma, a gramática especifica formalmente que uma atribuição pode conter tanto um valor simples quanto uma expressão aritmética, evidenciando como as produções definem a estrutura sintática válida da linguagem.

No desenvolvimento de linguagens, a gramática costuma ser o primeiro passo de implementação (MIGHT, [20--]) adaptando-se apenas a ferramentas utilizadas no projeto. Dessa forma, podemos concluir que uma gramática bem definida é o pontapé inicial para a construção de uma linguagem estruturada, pois permite formalmente a definição das sentenças que possuem sentido lógico na linguagem.

2.3 TABELA DE SÍMBOLOS

Para o desenvolvimento de tradutores de uma linguagem muitos recursos são utilizados. Um desses artifícios é a tabela de símbolos que nada mais é que uma estrutura de dados normalmente tabular que armazena informações sobre identificadores, constantes e assinaturas de funções de um código-fonte (AHO et al., 2007, p.07).

Figura 04 - Tabela de símbolos partido de um exemplo de código

Exemplo			
<pre>a :inteiro = 1 + 8; b :inteiro = a + 8; mostrarInteiro(b)</pre>			
Tabela de símbolos final			
Nome	Tipo	Endereço de memória	Escopo
a	inteiro	0x7FFF5	global
b	inteiro	0xL043	global

Fonte: A autora.

A Figura 04 mostra um exemplo de tabela de símbolos que armazena informações sobre as variáveis “a” e “b”. Estas informações são nome, tipo, endereços de memória do valor associado àquele identificador e o escopo que se encontra aquele

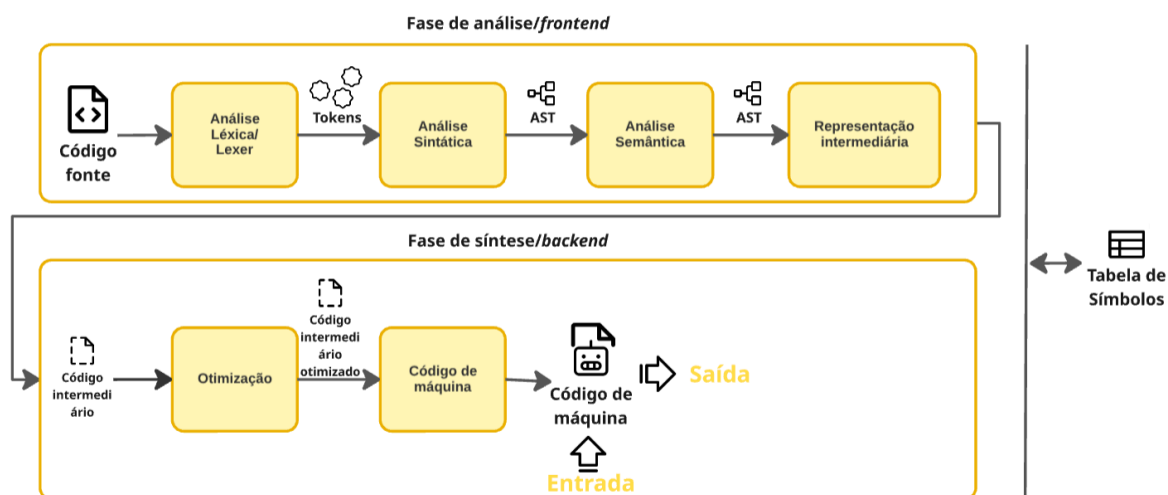
símbolo no texto-fonte. Em uma saída de dados via terminal da variável “a”, por exemplo, apenas com o seu nome é possível comunicar-se com a tabela de símbolos e ter acesso ao seu valor. Desse modo, é notável que com a utilização dessa estrutura de dados conseguimos acessar de forma simplificada metadados das variáveis e funções utilizadas no código e usá-los durante todo o processo de tradução (Sebesta, 2011, p.48).

3 COMPILADORES

Nessa seção, iremos abordar detalhadamente sobre as etapas de compilação de um texto-fonte, objetivando um aprendizado mais aprofundado sobre o que é necessário para a construção de uma linguagem de programação. Caso durante o processo de produção de linguagem o método de tradução optado pela equipe não for a compilação, o conhecimento apresentado nessa seção ainda será muito útil pois, assim como explicado anteriormente, há muitas etapas em comum entre as diferentes técnicas de tradução.

O processo de compilação de uma linguagem, assim como exemplificado no tópico 2.1.2, é um das formas de tradução de um código-fonte para binário de máquina. O compilador constitui dentro de si processos otimizados de transformação do código-fonte para linguagem de máquina, resultando também na rapidez de execução do programa. Por esse motivo, todos os exemplos construídos neste processo são centrados nas etapas de compilação que de acordo com Aho et al.(2007, p.03) são cinco: Análise léxica, análise sintática, análise semântica, representação intermediária, otimização e geração/execução de código de máquina. Além disso, um compilador pode ser dividido em 2 grandes módulos, que distribui as seis fases dentro deles, que é o módulo de análise ou *frontend* e módulo de síntese ou *backend*. A representação visual apresentada na Figura 05 mostra, de forma simplificada, esses módulos e fases.

Figura 05 - Módulos e fases de um compilador



Fonte: A autora adaptado de AHO et al.(2007, p.05).

3.1 ANÁLISE LÉXICA

A primeira etapa do processo de compilação é chamada de Análise Léxica ou Lexer. Partindo do código-fonte, o lexer tem como principal objetivo a geração de tokens, que podem ser definidos como a menor unidade de um programa.

Figura 06 - Tokens gerados a partir de uma código-fonte



Fonte: A autora.

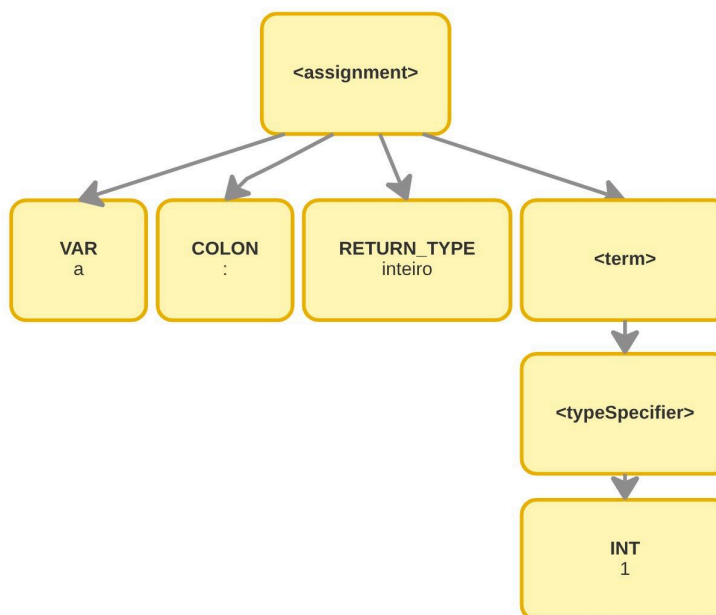
Após o processamento do código-fonte pela análise léxica, o texto é convertido em uma sequência de tokens, que representam unidades léxicas significativas da linguagem, como identificadores, palavras-chave, operadores e literais. A Figura 06 apresenta um exemplo prático em que, a partir da gramática simplificada composta pelos símbolos não-terminais *<assignment>*, *<term>* e *<typeSpecifier>*, e pelos

terminais *VAR*, *COLON*, *RETURN_TYPE*, *EQUAL* e *INT*, o código-fonte `a: inteiro = 1` é subdividido e classificado em seis tokens distintos: “a” é reconhecido como um token do tipo *VAR*, “:” como *COLON*, “inteiro” como *RETURN_TYPE*, “=” como *EQUAL* e “1” como *INT*. Esse processo evidencia que o analisador léxico tem como principal responsabilidade identificar e classificar os elementos do código-fonte, produzindo uma sequência de tokens que será utilizada nas etapas posteriores do processo de compilação.

3.2 ANÁLISE SINTÁTICA

Na análise sintática, a sequência de tokens produzida pelo analisador léxico é processada pelo parser para verificar se sua organização está de acordo com as regras definidas pela gramática da linguagem. Durante esse processo, é construída uma árvore sintática, na qual cada nó representa uma construção descrita pelas produções gramaticais, evidenciando de forma hierárquica como os tokens se agrupam para formar estruturas sintaticamente válidas (Sebesta, 2011, p. 200). Essa árvore constitui a base para as etapas posteriores do processo de compilação, como a análise semântica e a geração de código. Entretanto, a representação da árvore sintática pode variar conforme a implementação do compilador. Enquanto algumas implementações preservam todos os detalhes da derivação gramatical, outras utilizam uma Árvore Sintática Abstrata (Abstract Syntax Tree – AST), que elimina elementos puramente sintáticos, como delimitadores e produções intermediárias, preservando apenas as informações relevantes para representar a estrutura lógica do programa.

Figura 07 - Árvore sintática abstrata do exemplo da Figura 06



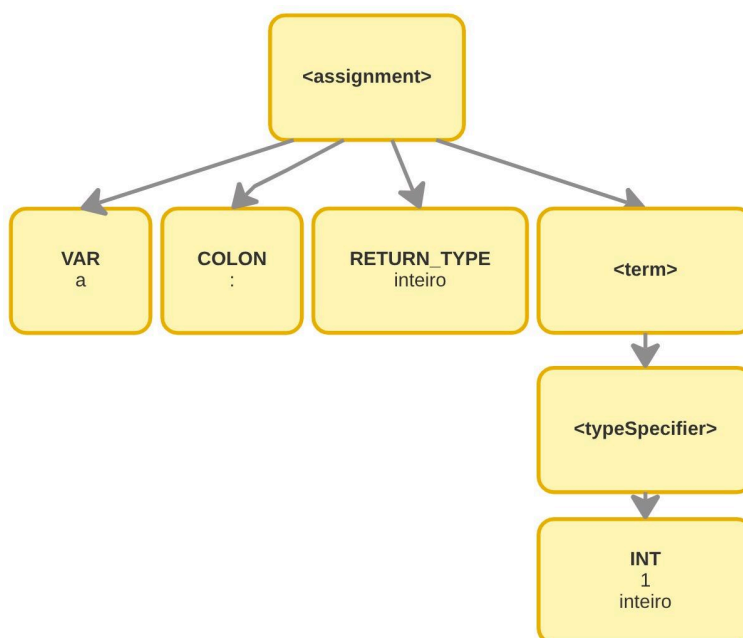
Fonte: A autora.

A Figura 07 apresenta uma AST construída a partir do código-fonte ilustrado na Figura 06. Nessa representação, cada nó corresponde a uma construção significativa da linguagem, organizada de forma hierárquica de acordo com a estrutura definida pela gramática, simplificando a manipulação da árvore nas etapas posteriores do compilador.

3.3 ANÁLISE SEMÂNTICA

Quando se trata da análise semântica é o momento de incrementar os nós da árvore sintática adicionando informações relevantes sobre tipagem, escopo e verificação se a estrutura sintática produzida pelo parser faz sentido em termos de significado dentro da linguagem (Aho et al., 2007, p.06).

Figura 08 - Árvore sintática abstrata do exemplo 07 após análise semântica



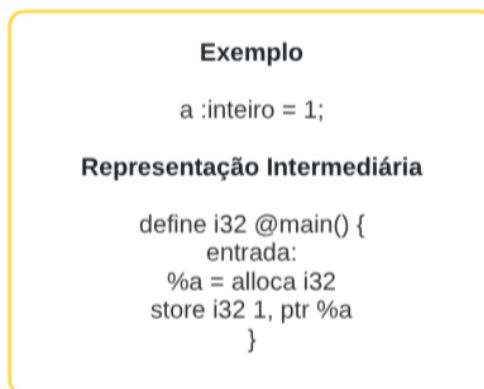
Fonte: A autora.

A Figura 08 mostra a AST criada na representação visual 07 após a realização da análise semântica. Como resultado foi feita a adição explícita do tipo do valor “1” que é “inteiro”. Além disso, nessa fase é verificado se o tipo definido da variável “a” no nó *RETURN_TYPE* é correspondente ao tipo associado ao valor “1”, que no caso ambos são inteiros. Ademais, comumente a análise semântica ocorre em paralelo com a análise sintática evitando processamento e criação da AST em casos de erros semânticos.

3.4 REPRESENTAÇÃO INTERMEDIÁRIA

Muitas implementações de compiladores utilizam, para auxiliar na transformação para código de máquina, a criação de uma representação intermediária (IR). Essa forma intermediadora é uma versão do código que intercepta as fases do frontend e backend de um compilador funcionando como uma abstração do código-fonte que preserva sua estrutura lógica, mas em um formato mais adequado para análise e transformação (Aho et al., 2007, p.06).

Figura 09 - Representação intermediária partindo da AST gerada no exemplo 08



Fonte: A autora.

A Figura 09 mostra um exemplo simplificado de uma representação gerada a partir da árvore sintática mostrada na seção anterior (3.3). No caso do tipo da IR do exemplo acima, sua linguagem se aproxima com algo similar a um assembly, em que um espaço de memória é alocado para a variável “a” e o valor “1” é armazenado nesse espaço de memória.

A importância de geração de uma IR e não diretamente de binários de máquina se justifica pelo aumento de formas de otimização do código original para que sejam aproveitados recursos como memória e chamadas de sistemas.

3.5 OTIMIZAÇÃO

Após a criação de uma representação intermediária o código resultante passa por uma série de otimizações. Elas refinam o código em diferentes níveis, fazendo melhorias significativas que antecedem a transformação para código de máquina (Aho et al., 2007, p.06). Dentre as otimizações listadas por Lima (2022) temos:

- Remoção de código morto, ou seja, comandos que não são utilizados em outras partes do código.
- Substituição de cálculos repetidos por referência aos resultados obtidos anteriormente para evitar processamento desnecessário.
- Otimização de laços de repetições.

Além das técnicas citadas acima existem outros meios para a melhoria da IR mas que costumam ser abstraídas por ferramentas que produzem o backend do compilador.

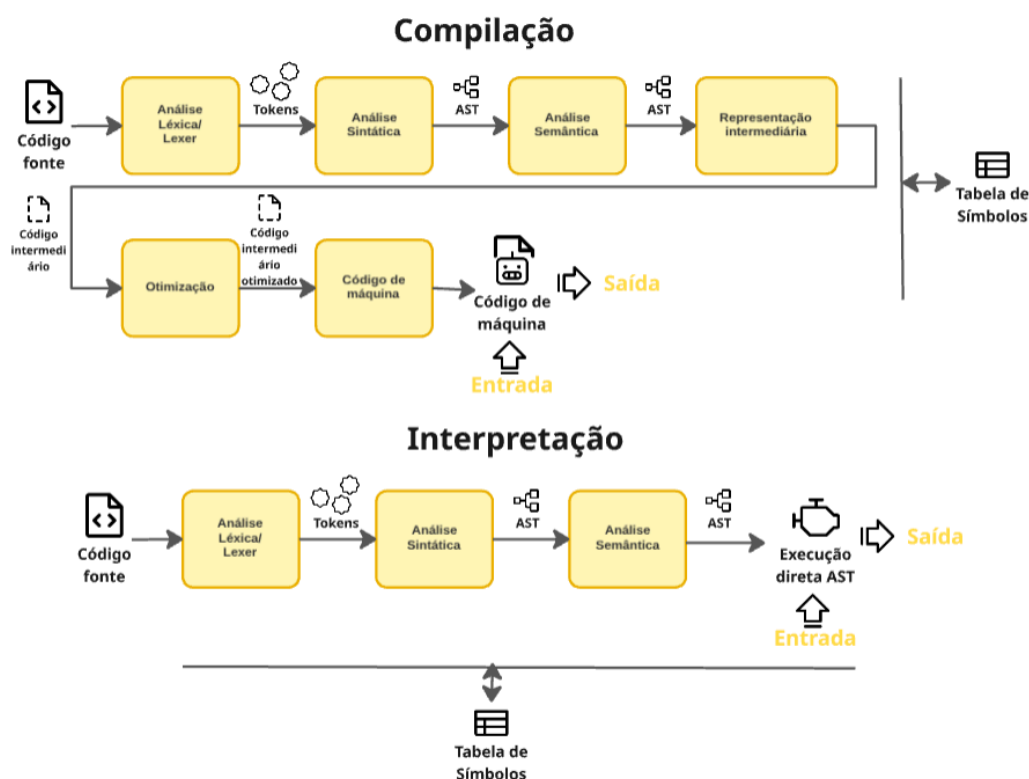
3.6 CÓDIGO DE MÁQUINA

Por fim, a etapa que antecede o fim do ciclo de compilação é a geração de código de máquina. Nessa fase a representação intermediária que passou por etapas de otimização é transformada em instruções binárias diretamente compreensíveis pelo hardware (Aho et al., 2007, p.07).

Após a geração do código de máquina, o compilador disponibiliza ao programador o executável resultante, que representa o produto final do processamento do texto-fonte. Dessa forma, partindo do executável o programador pode processar e obter as respostas para o seu programa.

Portanto, podemos compreender a compilação como um processo complexo e com etapas bem definidas. Além disso, conseguimos delimitar agora, de forma clara, as diferentes etapas dos métodos de tradução de um código, como interpretador e compilador e entender a diferença entre eles.

Figura 10 - Comparação entre as etapas da compilação e interpretação



Fonte: A autora adaptando de Aho et al. (2007).

Na Figura 10 é notável a diminuição de etapas que um interpretador realiza em comparação com o compilador pois as etapas de representação intermediária, otimização e geração de código de máquina são eliminadas pelo interpretador. Isso se explica pelo fato de que o interpretador executa diretamente os comandos (Lee 2017) .

Dessa forma, conseguimos delimitar as principais diferenças dos métodos de translação e entender detalhadamente as etapas de compilação para que seja possível iniciar a produção da linguagem.

4 FERRAMENTAS UTILIZADAS

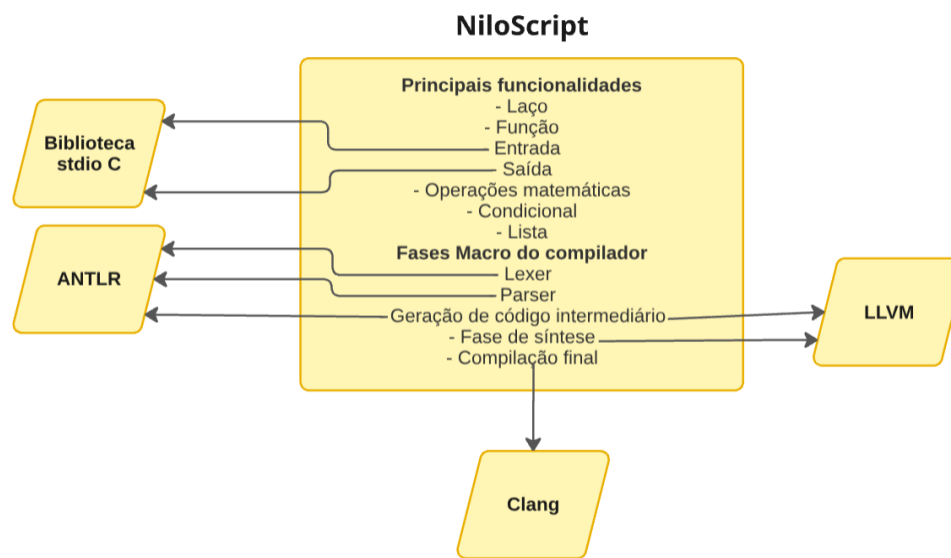
Este processo está estruturado em fases, cada uma acompanhada de uma seção prática para ilustrar o funcionamento da mesma. Para fins de demonstração, será utilizada uma linguagem desenvolvida especificamente para este processo, denominada NiloScript. Contudo, isso não implica que as ferramentas empregadas em conjunto para a criação desta linguagem devam necessariamente ser utilizadas nas implementações. Todavia, para a compreensão dos exemplos práticos, é importante entender a utilização das ferramentas pois isso corrobora na escolha de instrumentos que se adequem às necessidades do projeto.

4.1 LINGUAGEM NILOSCRIPT

Aprender sobre um novo tópico pode ocorrer de maneiras diferentes. De acordo com o método *Vark* de estudo, existem 4 formas de aprendizagem que incluem estímulos visuais, auditivos, leitura/escrita e cinestésica. Esse processo busca estimular a aprendizagem focando na utilização de métodos de ensino visuais e cinestésicos (Vaek, 2026). Para exemplificações práticas serão empregados incentivos gráficos, isso significa que para cada seção de atividades apresentadas em cada fase do processo, iremos mostrar, de forma visual e prática, como pode ser realizada cada atividade proposta, utilizando como objeto de exemplificação, a linguagem de programação NiloScript.

O NiloScript é uma linguagem de programação de propósito geral (GPL) simplificada produzida com C++ criada especificamente para este trabalho. A linguagem surgiu com o objetivo de demonstração simplificada de estruturas básicas como: Laços de repetição, funções, listas, operações aritméticas e outras estruturas simples que compõem normalmente as linguagens de propósito geral.

Figura 11 - Estruturas e ferramentas do NiloScript.



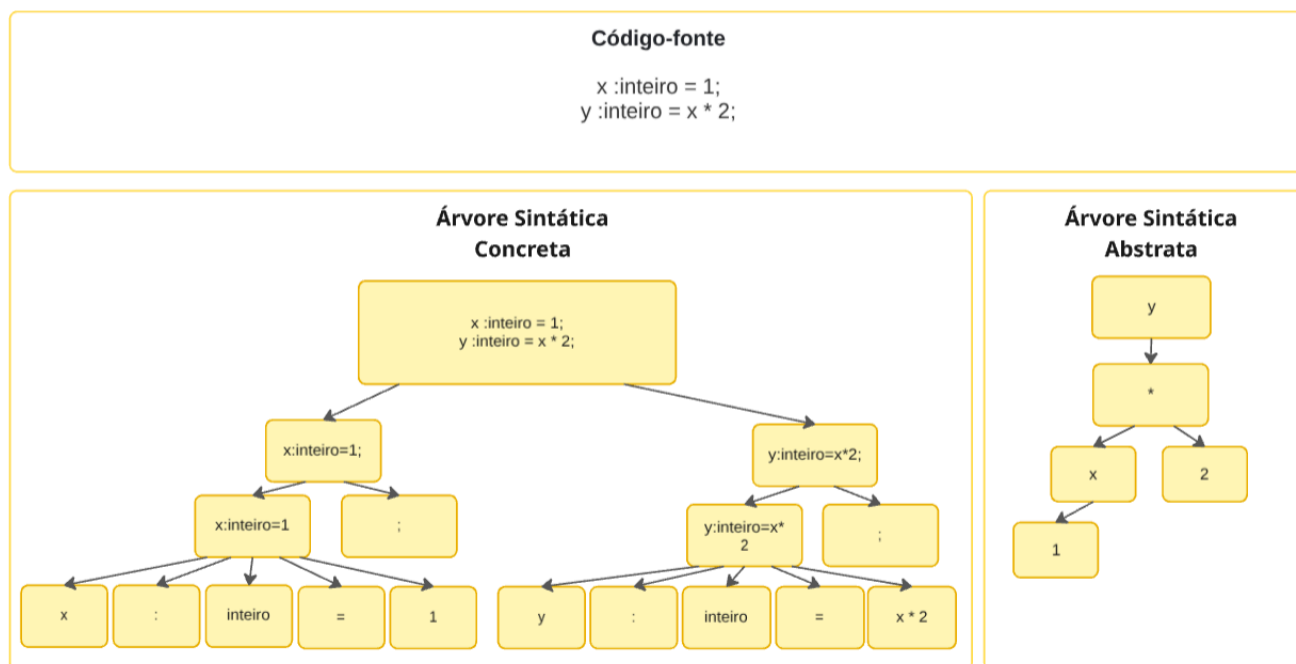
Fonte: A autora.

Para a produção da linguagem foram utilizadas diferentes tecnologias de apoio, com o objetivo de simplificar a produção e melhorar a legibilidade dos exemplos apresentados no processo. Conforme ilustrado pela Figura 11, tais tecnologias desempenharam papéis distintos e complementares na composição da linguagem, em que é válido evidenciar o uso do ANTLR e LLVM. Essas ferramentas auxiliam na construção do compilador da linguagem e serão detalhados posteriormente, de modo a evidenciar a integração ao projeto e as contribuições específicas para a estruturação da linguagem NiloScript.

4.2 ANTLR

O ANTLR (ANother Tool for Language Recognition) é um gerador de *parser* compatível com linguagens como Java, C++, Python e assim por diante. Ele possibilita a criação de árvores sintáticas concretas (CST) baseando-se em um arquivo de gramática (Parr, 2013). Isso significa que o ANTLR a partir da definição de símbolos terminais e não-terminais definidos pelo programador criador da linguagem gera tokens por meio da análise léxica e em seguida gera a árvore na análise sintática.

Figura 12 - Comparação de uma árvore sintática abstrata e uma concreta de um código-fonte NiloScript.



Fonte: A autora.

A Figura 12 demonstra a diferença de uma CST e uma AST, que se caracteriza no nível de detalhamento dos tokens. Apesar de os nós da árvore sintática concreta gerada possuírem informações minuciosas sobre o código-fonte, não há a realização de análise semântica por parte da ferramenta. Por esse motivo a linguagem *NiloScript*, utilizada neste modelo de processo como um auxiliar representativo, em sua implementação gera uma árvore abstrata (AST) partindo da CST do ANTLR para aumentar a segurança e confiabilidade da linguagem a partir de uma análise semântica detalhada.

Embora seja uma ótima ferramenta, que simplifica as etapas de análise de um compilador por cumprir o papel de *lexer* e *parser*, o *ANTLR* não é a única ferramenta que possibilita o desenvolvimento dessas etapas do compilador de uma linguagem. A seguir, iremos analisar algumas tecnologias que também podem ser utilizadas como substitutas.

Quadro 02 - Tecnologias que podem substituir o ANTLR

Tecnologia	Descrição	O que realiza	Linguagens que prestam suporte
Flex	Gera tokens a partir de expressões regulares.	Lexer	C/C++

Bison	A partir de uma gramática formal e tokens gerados pelo flex ele constrói uma árvore sintática que pode ser utilizada como base na construção de árvores sintáticas abstratas. Pode realizar análises semânticas a partir de ações definidas na gramática.	Parser e análise semântica	C/C++
Yacc	Assim como o bison gera um analisador sintático em formato de tabela a partir de tokens fornecidos pelo flex.	Parser	C
Javacc	Partindo de descrições de gramática em Formato EBNF ele gera código Java puro.	Lexer e Parser	Java

Fonte: A autora.

Além das ferramentas apresentadas no Quadro 02, há outras possibilidades viáveis no mercado, cabe à equipe de desenvolvimento decidir a melhor escolha para a implementação do projeto.

4.3 LLVM

O *LLVM project* é uma infraestrutura de compiladores reutilizáveis. Criado em 2000 e produzido em C++ ele foi projetado com o propósito de desenvolver técnicas de compilação estática e dinâmica (Llvm Project, 2026).

No desenvolvimento da linguagem NiloScript, optou-se pela utilização de uma compilação estática em que o LLVM foi o responsável pela fase de síntese e geração de representação intermediária. Em outras palavras, ele serviu de suporte para que a

partir da árvore sintática concreta gerada pelo ANTLR e adaptação para a geração de uma AST, fossem criadas representações intermediárias para os códigos-fonte fornecidos. A partir da representação intermediária o CLANG atuou como orquestrador de compilação das etapas de otimização, geração de código e ligação, delegadas à infraestrutura do LLVM (Project, 2026). Porém, apesar do LLVM ser uma ferramenta completa e complexa, motivando a escolha dele para a produção da linguagem modelo NiloScript, há outras opções possíveis que podem desempenhar um papel similar.

Quadro 03 - Tecnologias que podem substituir o LLVM

Tecnologia	Descrição	O que realiza	Linguagens que prestam suporte
GNU Compiler Collection (GCC)	Conjunto de compiladores que cobre quase todas as fases de compilação.	Lexer, parser, otimização e geração de código	C, C++, Fortran, Ada, Go, entre outras.
QBE	Atua como backend eficiente para compiladores.	Geração de código intermediário e otimização.	C
Tiny C Compiler (TCC)	Apesar de não possuir otimizações avançadas como LLVM ou GCC, mas cobre as fases essenciais para transformar código-fonte em executável.	Análise e geração de código.	C

Fonte: A autora.

Tal qual o apresentado no Quadro 03 há diferentes tecnologias que podem prestar suporte similar ao LLVM. Porém, assim como toda tecnologia há seus pontos de diferenciações, por isso cabe aos programadores a decisão de qual irá suprir de forma otimizada as necessidades do seu projeto.

5 FASES DO NILO

Com o objetivo de descomplicar a compreensão e desenvolvimento da linguagem iremos dividir o NILO em cinco fases. Essas etapas possuirão o seu

propósito, artefatos produzidos e por fim uma atividade prática com exemplos de possíveis resoluções.

Porém, apesar da exposição linear das fases é importante ressaltar que a aplicação prática desse modelo de processo ocorre de forma iterativa, ou seja, não há restrições quanto ao retorno para outras fases em contextos que necessite, por exemplo, a adição de novas funcionalidades ou documentação da linguagem.

Figura 13 - Fases do modelo de processo

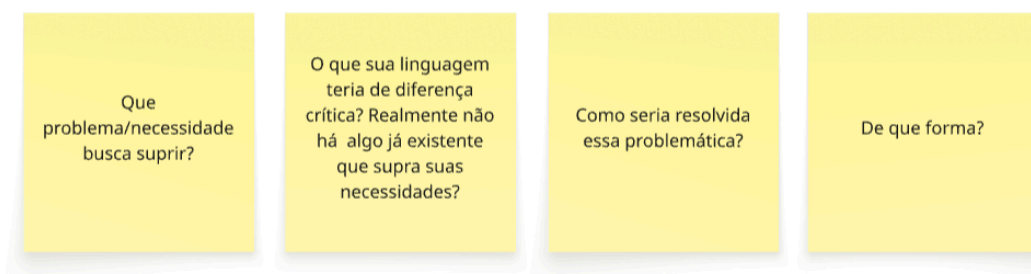


Fonte: A autora.

5.1 RECONHECIMENTO

Para a produção de uma nova linguagem de programação é essencial examinar as necessidades que cercam a sua criação. Inicialmente, antes de desenvolver iremos levantar as características para compreender o que é requerido por meio de questionamentos.

Figura 14 - Questionamentos necessários na fase de reconhecimento



Fonte: A autora.

A Figura 14 representa quais vão ser os questionamentos que devem ser respondidos na fase de reconhecimento para gerar ao final uma síntese do escopo e requisitos da linguagem. A seguir, vamos destrinchar a motivação de cada pergunta:

Que problema/necessidade busca suprir?

Primeiramente, o surgimento de uma nova linguagem de programação normalmente vem acoplada com a ideia de suprimir um problema que não pode ser resolvido com linguagens de propósito geral (GPL) , ampliando uma necessidade de desenvolvimento de uma de domínio específico.

Então, inicialmente é fundamental definir o problema real que cerca a ideia de desenvolvimento de uma nova linguagem, podendo ser pela ausência de suporte de outras ferramentas ou até mesmo por demandas pessoais. A questão é definir a adversidade, para que assim possamos pensar em como resolver.

O que sua linguagem teria de diferença crítica? Realmente não há algo já existente que supra suas necessidades?

Desenvolver uma linguagem de programação mesmo com o uso de ferramentas que dão o suporte para isso não é uma tarefa simples, por isso é importante através de buscas, analisar se o que você necessita não é feito por outras linguagens. Nesse momento é essencial o estabelecimento e retenção dos diferenciais que colaborarão na resposta da próxima pergunta.

Como seria resolvida essa problemática?

Neste instante é momento de refletir sobre os atributos presentes em uma linguagem que supra o problema definido. Por exemplo, ela deve ter acesso à memória? Deve ter suporte a bibliotecas externas? É necessário examinar essas questões.

Respondendo esta pergunta será possível definir de forma clara qual o escopo geral da linguagem e qual papel ela vai desempenhar na diminuição da problemática apontada na pergunta inicial.

De que forma?

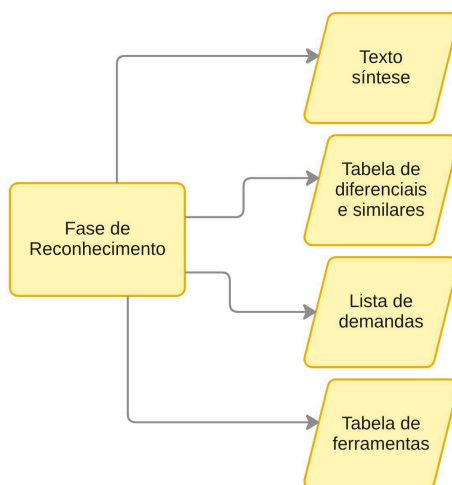
Esta parte é destinada para aqueles indivíduos que já possuem alguma expertise com a produção de linguagens ou buscam testar conhecimentos aprendidos que até agora tinham sido vistos apenas na teoria, pois é preciso definir as ferramentas que serão usadas em todo o processo de desenvolvimento. Nas partes práticas serão utilizadas as ferramentas citadas

anteriormente, mas há à flexibilidade de escolha dos instrumentos que se adequam ao desenvolvimento da sua linguagem. Contudo, tal fato não renega a realização dos exercícios de treinamento.

5.1.1 Mão na massa

O seu propósito na fase de reconhecimento é responder os questionamentos levantados no tópico anterior, gerando artefatos em cada questionamento. O processo de análise e de respostas às perguntas deve abranger todos os interessados e envolvidos na criação da nova linguagem para que assim o escopo fique bem definido.

Figura 15 - Artefatos que devem ser produzidos na fase de reconhecimento



Fonte: A autora.

A imagem acima representa visualmente os artefatos que devem ser produzidos até o fim da fase de reconhecimento, para documentação e organização das ideias.

Quadro 04 - Tabela de questionamento x artefato gerado

Nº	Pergunta	Objetivo	Artefato gerado
01	Que problema/necessidade busca suprir?	Compreender os obstáculos gerados pela ausência de uma linguagem que supra as necessidades.	Texto síntese.
02	O que sua linguagem teria de diferença crítica?	Justificar o “porquê” da criação de uma nova	Tabela de similares

	Realmente não há algo já existente que supra suas necessidades?	linguagem, partindo da busca de ferramentas similares e do relato dos diferenciais.	
03	Como seria resolvida essa problemática?	Definir as características necessárias para a linguagem a ser desenvolvida.	Lista de demandas
04	De que forma?	Enumerar as ferramentas que serão empregadas para a realização dos objetivos, detalhando a funcionalidade da escolha.	Listagem das ferramentas e suas respectivas funcionalidades

Fonte: A autora.

Cada questionamento gera um artefato documental (texto síntese, tabela comparativa, lista de demandas, tabela de ferramentas), que serve como registro formal e estruturado das decisões tomadas, ajudando a assegurar que o desenvolvimento posterior ocorra de forma alinhada às necessidades dos interessados.

5.1.2 Exemplo prático

A partir da aplicação da pergunta 01 (Figura 14) com todos os envolvidos no desenvolvimento da linguagem, o artefato sugerido para ser produzido logo em seguida é o **Texto Síntese**. Nele deve ser definido de forma clara e concisa o problema que motivou a ideia de criar uma linguagem. A Figura abaixo mostra um exemplo prático de Texto Síntese da linguagem de programação NiloScript, que será utilizada ao longo dos exemplos de cada artefato para a compreensão clara do que deve ser produzido.

Figura 16 - Exemplo de texto síntese do *NiloScript*

Os estudantes do curso de Análise e Desenvolvimento de Sistemas do IFRN apresentam lacunas significativas na formação básica em linguagens de programação. Diante desse cenário, pretende-se desenvolver uma linguagem com caráter educativo, denominada NiloScript, que pretende oferecer suporte didático e prático ao processo de aprendizagem. Como se trata de uma iniciativa voltada ao ensino, a linguagem precisa adotar uma forma mais natural e acessível, optando pelo uso do português para aproximar o estudante do código e reduzir barreiras de compreensão. No entanto, essa naturalidade não deve se afastar dos conceitos fundamentais de programação, pois é justamente a fidelidade a esses princípios que garante a aplicabilidade prática e a transferência de conhecimento para outras linguagens.

Além disso, a ausência de uma linguagem como o NiloScript gera obstáculos significativos no processo de ensino. Sem uma ferramenta que traduza conceitos técnicos para uma forma mais natural e acessível, os estudantes tendem a enfrentar dificuldades na compreensão das etapas que envolvem a construção de compiladores, permanecendo distantes de práticas fundamentais como análise léxica, sintática e geração de código. Portanto, a falta de um recurso pedagógico estruturado em português pode ampliar a barreira linguística, tornando o aprendizado mais abstrato e menos inclusivo.

Fonte: A autora.

Para o questionamento 02 (Figura 14) o objeto gerado é a **Tabela de Similares**. Nessa tabela é associada a cada linguagem mencionada as características que se ausentam mas que seriam essenciais na sua linguagem para resolver o problema apresentado. A partir dessa análise será definida a real necessidade de criação de uma nova linguagem, refletindo se nenhuma das linguagens pesquisadas cumprem as necessidades do projeto.

Quadro 05 - Exemplo de tabela de similares do *NiloScript*

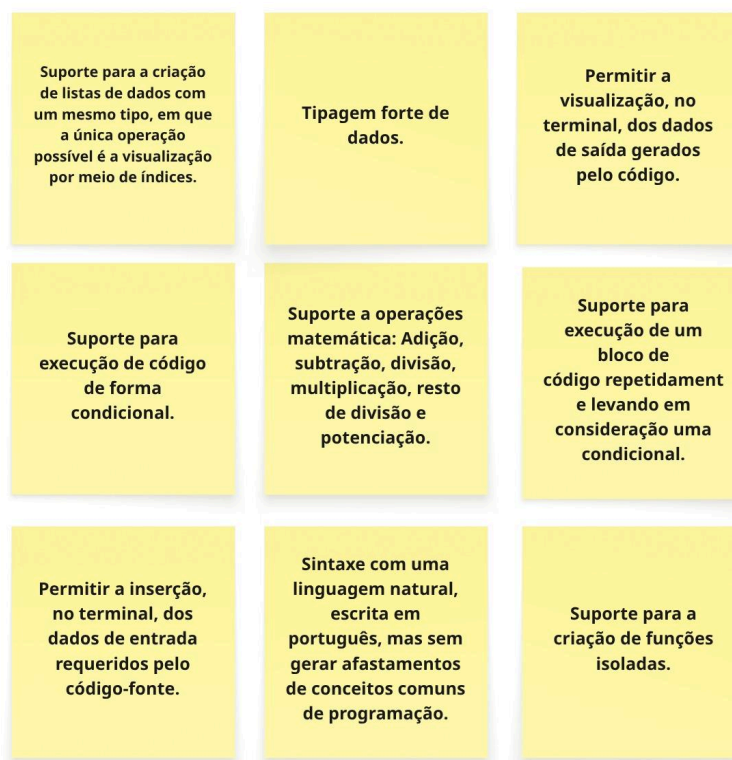
Linguagem	Descrição	Similaridades que minha linguagem deve ter	Diferenciais que minha linguagem deve ter
Potigol	Linguagem funcional criada no IFRN, voltada para ensino. Tem sintaxe simples e clara, ideal para introduzir conceitos de programação funcional sem complexidade excessiva.	Os comandos serem em português.	- Compilador próprio que faça de forma independent e a tradução. - Tipagem obrigatória
Scratch	Permite que	Ter um teor	Comandos

	iniciantes aprendam lógica de programação sem precisar escrever código textual. Muito usada em escolas para introduzir conceitos de algoritmos e pensamento computacional.	didático	em português. Utilização de linhas de código para produzir programas.
--	--	----------	--

Fonte: A autora

Na pergunta de número 03 temos como material resultante a **Lista de Demandas**. Essa listagem elenca o que é necessário na linguagem para resolver o problema apresentado no texto síntese (Figura 16).

Figura 17 - Exemplo de listagem de demandas de funcionalidades presentes no *NiloScript* para resolver o problema apresentado



Fonte: A autora

Para a última pergunta(Figura 14) é requerido o desenvolvimento da **Listagem das Ferramentas** que irão prestar suporte na implementação de todas as funcionalidades apontadas na listagem de demandas (Figura 17).

Quadro 06 - Exemplo da tabela de ferramentas necessárias para desenvolver o *NiloScript*

Ferramenta	Funcionalidade
ANTLR	Produção da gramática, seguida pela construção do lexer e do parser, resultando ao final em uma árvore sintática concreta, adequada para servir como base na geração de código intermediário.
LLVM	Gerador de representação de código intermediário.

Fonte: A autora

Embora existam indicações de perguntas consideradas essenciais para esta fase, recomenda-se a inclusão de outras questões conforme as necessidades específicas do projeto. É fundamental que a produção de artefatos ocorra a cada pergunta realizada, de modo a sintetizar as informações coletadas e assegurar sua utilidade e aplicabilidade na etapa seguinte do desenvolvimento.

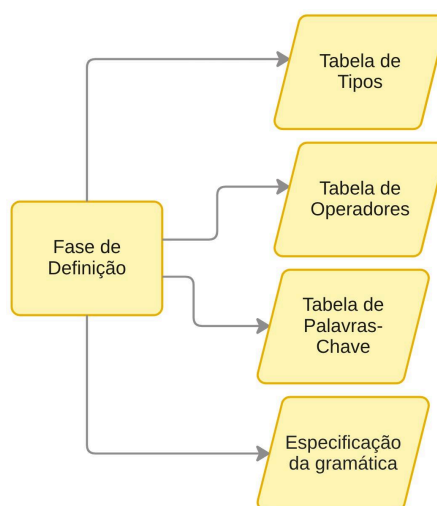
5.2 DEFINIÇÃO

A segunda fase, denota-se pela definição das características necessárias da linguagem para cumprir o que foi definido como domínio, partindo dos artefatos produzidos na etapa anterior. Durante todo o processo, a consulta e análise de todos os artefatos gerados são essenciais, pois são neles que contém informações importantes de necessidades da linguagem que foram observadas.

5.2.1 Mão na massa

Sua tarefa nesta fase é produzir os artefatos: **Tabela de Tipos, Tabela de Operadores, Tabela de Palavras-Chave e Especificação da gramática.**

Figura 18 - Artefatos a serem produzidos na fase de definição



Fonte: A autora.

No primeiro artefato, deve ser estabelecido o nome e uma descrição sobre cada tipo de dado, mas ele deve ser produzido apenas se a linguagem possuir tipagem de dados. Não obstante, a tabela de operadores será utilizada para definir quais são os símbolos reservados da linguagem e seus respectivos papéis. Além disso, a definição das funcionalidades completas da linguagem, como funções, condicionais e laços de repetição, será realizada por meio do artefato de Especificação da Gramática. Esse artefato representa a estrutura sintática das construções da linguagem, estabelecendo regras que descrevem como os programas podem ser escritos. Para sua elaboração, são utilizados os elementos definidos nos artefatos anteriores, como os tipos de dados, operadores e palavras-chave, sendo sua especificação frequentemente representada por meio da notação Backus–Naur Form (BNF) ou de suas variações. Por último, a tabela de palavras-chave irá detalhar quais caracteres ou palavras, além dos tipos e operadores, possuem um significado específico na linguagem, não podendo ser utilizados como identificadores de variáveis, listas e funções.

5.2.2 Exemplo prático

A primeira demonstração consistirá na apresentação da tabela de tipos, na qual cada nome deverá corresponder exatamente ao identificador utilizado na linguagem. A respectiva descrição deverá indicar explicitamente quais categorias de dados podem ser armazenadas ao se empregar determinado tipo.

Quadro 07 - Exemplo da tabela de tipos simplificada do NiloScript

Nome do Tipo	Descrição
Inteiro	Tipo primitivo utilizado para representar valores numéricos inteiros, ou seja, números sem parte fracionária.
Flutuante	Tipo numérico destinado à representação de valores reais, ou seja, números que possuem parte fracionária.
Caracter	Tipo destinado a representação de cadeia de caracteres, ou seja, para representar textos.
Bool	Tipo destinado a representação de valores de verdadeiro ou falso.
Nada	Tipo destinado a representação da ausência de valor especificamente no contexto de retorno de funções.

Fonte: A autora.

A seguir é necessário listar quais os operadores reservados e suas respectivas definições e funções semânticas no contexto da própria linguagem.

Quadro 08- Exemplo da tabela de operadores do NiloScript

Operador	Descrição
;	Representa fim de comandos.
*	Representa a operação matemática de multiplicação.
/	Representa a operação matemática de divisão.
%	Representa a operação matemática de resto de divisão.
**	Representa a operação matemática de potenciação.
+	Representa a operação matemática de adição.
-	Representa a operação matemática de subtração.

>	Representa o operador de comparação maior que.
<	Representa o operador de comparação menor que.
>=	Representa o operador de comparação maior ou igual que.
<=	Representa o operador de comparação menor ou igual que.
!=	Representa o operador de comparação diferente de.
==	Representa o operador de comparação igual que.
{ }	Utilizado para conter bloco de código executado segundo alguma condição.
()	Utilizado para priorizar a execução de operações matemáticas, para conter condicionais em laços de repetição ou em condicional ou na definição de parâmetros de funções.
[]	Utilizado para acesso a elementos de uma lista por meio do índice.
:)	Indica que aquela linha de código será ignorada durante o seu processamento.

Fonte: A autora.

Como exemplo de aplicação do artefato de Especificação da Gramática, a Figura 19 apresenta um trecho da gramática da linguagem NiloScript representado na notação Backus–Naur Form (BNF). A especificação da gramática descreve formalmente a estrutura sintática da linguagem por meio de produções que definem como seus elementos podem ser combinados para formar programas válidos. Embora a Figura 12 utilize a notação BNF, outras notações podem ser empregadas para representar uma gramática.

Figura 19 - Especificação da gramática utilizando a notação BNF

```

<program> ::= { <stmt> }
<stmt> ::= <function>
<function> ::= "funcionalidade" VAR
              "(" [ <parameter> { "," <parameter> } ] ")"
              ":" RETURN_TYPE
              "{"
                { <stmt> }
                [ "retorne" <expression> ";" ]
              "}"
<parameter> ::= VAR ":" RETURN_TYPE
<expression> ::= <term>
<term> ::= VAR | INT | FLOAT | STRING | BOOL

```

Fonte: A autora.

A elaboração da especificação da gramática do NiloScript, foi realizada no ANTLR, na qual a gramática é descrita em um arquivo com extensão .g4. O nome desse arquivo é definido de acordo com a linguagem implementada, para o NiloScript, foi utilizado o arquivo NiloScript.g4.

Para determinar a qual linguagem aquela gramática pertence é importante colocar o indicativo no início do arquivo “*grammar*” e o nome da linguagem. Posteriormente, é o momento de definir as regras léxicas e sintáticas. As regras léxicas são aquelas que geram tokens e representam os terminais da linguagem como por exemplo na Figura 20 o terminal “VAR” que representa variáveis que podem ser palavras de A a Z minúsculas ou maiúsculas e com ou sem uso de sublinhado (_). As regras sintáticas de uma gramática são aquelas que representam não-terminais da linguagem, ou seja, regras que simbolizam estruturalmente cada funcionalidade da linguagem utilizando-se de outros terminais e não-terminais definidos na linguagem.

Figura 20 - Trecho da gramática do NiloScript

```

grammar NiloScript;

expression : VAR EQUAL (term | acessList | functionCall | input);

VAR : [a-zA-Z_][a-zA-Z0-9_]*;

```

Fonte: A autora.

A Figura 20 apresenta a regra sintática *expression*, composta pelo terminal *VAR*, pelo terminal *EQUAL*, que representa o símbolo de atribuição (=), e por um grupo de

alternativas que pode corresponder aos não-terminais *term*, responsável pelas operações matemáticas, *acessList*, referente ao acesso a elementos de uma lista, *functionCall*, que representa uma chamada de função, ou *input*, utilizado para a leitura de dados. Outra característica da gramática é a convenção de nomenclatura adotada, em que as regras sintáticas (não-terminais) são escritas em letras minúsculas, enquanto as regras léxicas (terminais) são representadas em letras maiúsculas. A gramática completa da linguagem está disponível no repositório oficial do projeto no GitHub, por meio do seguinte link: <https://github.com/namariaa/Nilo/blob/main/NiloScript/src/antlr/NiloScript.g4>.

Quadro 09 - Utilitários para montagem de gramáticas no ANTLR

Símbolo	Significado	Exemplo no NiloScript
*	Uma estrutura pode se repetir 0 ou muitas vezes.	VAR : [a-zA-Z_][a-zA-Z0-9_]*; Ou seja, uma variável é um caractere que pode ou não ser seguido por outros caracteres. Isso permite que uma variável seja “a” ou “primeiraVarivavel” ou até mesmo “media_ponderada”.
+	Uma estrutura deve se repetir 1 ou muitas vezes.	INT : [0-9]+; Ou seja, um número inteiro é pelo menos um número que pode ser alongado para a junção de vários números. Isso permite a formação de valores como 1 ou até mesmo 111.
~	Negação de uma estrutura.	STRING : '"' ~('"') * '"'; Ou seja, uma string pode ser qualquer coisa menos uma segunda aspas pois isso representaria o fim da string. Isso possibilita criação de strings como “olá, mundo” desde que não seja “olá “mundo” pois há uma aspas a mais.
EOF	Indica o fim do arquivo.	program : (stmt)+ EOF; Ou seja, um programa é uma lista de códigos até o fim do arquivo.
-> skip	Indica que toda vez que o lexer encontrar aquela estrutura ela vai ser	TAB : [\t]+ -> skip; Ou seja, toda vez que ele encontrar uma tabulação no código-fonte ele irá

	ignorada e não gerará tokens e consequentemente não aparecerá na árvore.	ignorar. Isso significa que: “olá, mundo” e “olá, mundo” São a mesma coisa para o ANLTR.
--	--	---

Fonte: A autora.

Além disso, o ANTLR fornece utilitários para montagem das gramáticas. O Quadro 15 mostra alguns símbolos e palavras-chave que podem ser utilizadas durante a montagem da gramática para auxiliar e facilitar a indicação para a ferramenta de como algo deve ser feito. Um exemplo disso é o símbolo de * (asterisco) que quando usado na sua gramática juntamente com não-terminais o parser do ANTLR monta a árvore com aquela estrutura em específico sendo uma lista que pode conter nenhum elemento ou vários.

Por fim, a tabela de palavras-chave deve conter todos os termos que possuem significado na linguagem. No documento produzido para o NiloScript, representado na imagem abaixo, foram incluídas todas as palavras que não simbolizam nenhuma funcionalidade da linguagem, pois essa exemplificação ficou a cargo do artefato anterior (Quadro 09).

Quadro 10- Exemplo da tabela de palavras-chave da linguagem NiloScript

Palavra-chave	Descrição	Exemplo de uso
pegaInteiro	Representa uma chamada de números inteiros via terminal.	a :inteiro = pegaInteiro ;
pegaFlutuante	Representa uma chamada de números com ponto flutuante via terminal.	a :flutuante = pegaFlutuante ;
pegaCaracteres	Representa uma chamada de caracteres via terminal.	a :caracter = pegaCaracteres ;
mostrarInteiro	Representa a exibição de números inteiros via terminal.	mostrarInteiro(a) ;
mostrarFlutuante	Representa a exibição de números com ponto flutuante via terminal.	mostrarFlutuante(a) ;
mostrarCaracteres	Representa a exibição de caracteres via terminal.	mostrarCaracteres(a) ;

mostrarBool	Representa a exibição de booleanos via terminal.	mostrarBool(a);
caso	Representa a funcionalidade de condicional.	caso (condicao){ o que vai ser realizado }
senao	Representa o início de um bloco de código caso uma condição seja falsa em uma funcionalidade de condicional.	senao { o que vai ser realizado }
retorne	Representa a palavra que antecede o que deve ser retornado em uma função.	retorne valor_retornado;
verdadeiro	Representa o valor verdadeiro.	a :bool = verdadeiro ;
falso	Representa o valor falso.	a :bool = falso ;
:)	Representa que uma linha está comentada.	:) Esse é um comentário
enquanto	Representa um laço de iteração.	enquanto (a > b) { o que vai ser realizado}
funcionalidade	Representa uma função.	funcionalidade nome_função (nome_parametro :tipo_parametro) :tipo_retorno_funcao { o que vai ser realizado retorne valor_retornado; }

Fonte: A autora.

5.3 ESTRUTURAÇÃO

Um passo crucial que antecede o início de projetos de implementação é a tomada de decisões projetuais. Tais determinações englobam definições de padrões de

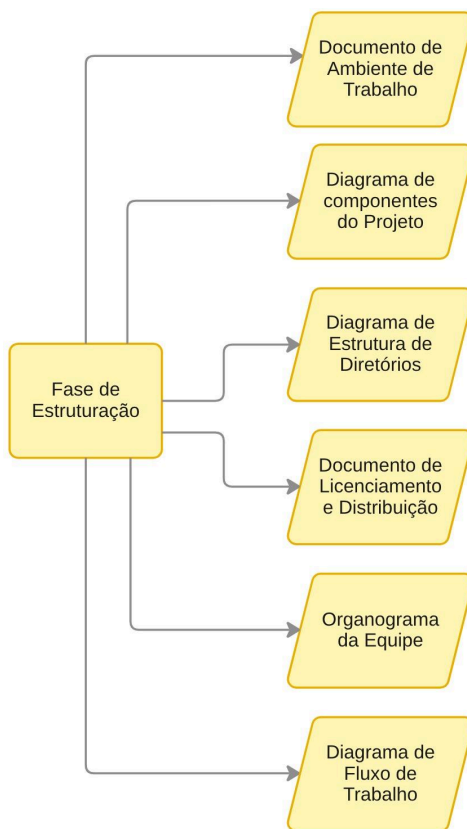
desenvolvimento e perpassam pela organização de equipe até questões jurídicas, em casos de grandes projetos corporativos.

A estruturação é composta pela produção de artefatos que suportam projetos de diferentes portes. Portanto, é importante decidir quais artefatos adequados ao seu projeto antes do início da produção.

5.3.1 Mão na massa

O processo de codificação deve ser antecipado pela adequação do ambiente de trabalho e pela organização da equipe. Dessa forma, a sua tarefa nesta fase é a criação de artefatos que auxiliem nessa coordenação que são: **Documento de Ambiente de Trabalho**, **Diagrama de Arquitetura do Projeto**, **Diagrama de Estrutura de Diretórios**, **Organograma da Equipe**, **Diagrama de Fluxo de Trabalho** e o **Documento de Licenciamento e Distribuição**.

Figura 21 - Artefatos que devem ser produzidos na fase de estruturação



Fonte: A autora.

O primeiro artefato a ser produzido consiste em um documento de organização do ambiente de trabalho, cuja finalidade é registrar os recursos utilizados no projeto. Nele deve ser delimitado o local de armazenamento e versionamento de código, ambiente de desenvolvimento integrado (IDE), o sistema operacional utilizado e qualquer informação relevante para a construção do ambiente de produção da linguagem.

Quadro 11 - Ferramentas de versionamento e armazenamento de código

Ferramenta	Diferencial
Github	Plataforma baseada em Git utilizada para hospedagem e versionamento de código. Destaca-se pela ampla gama de ferramentas de integração contínua.
Gitlab	Plataforma de versionamento de código que incorpora nativamente funcionalidades de CI/CD e gestão de repositórios.
Bitbucket	Plataforma de armazenamento de repositórios Git com foco em ambientes corporativos. Fornece recursos avançados de controle de versão e gestão de equipes.

Fonte: A autora.

Quadro 12 - Ambientes de desenvolvimento integrados (IDE)

IDE	Diferencial
Visual Studio Code	Editor de código com suporte a diversas linguagens e integração com sistemas de versionamento. Além de permitir a personalização avançada do ambiente de desenvolvimento.
Eclipse	IDE robusta adequada para aplicações Java de grande porte. Oferece amplo ecossistema de plugins, suporte a múltiplas linguagens e ferramentas de modelagem.
Intellij	Ambiente de desenvolvimento profissional, reconhecido pelo suporte

	às linguagens Java e Kotlin. Amplamente utilizado em contextos industriais.
--	--

Fonte: A autora.

Os Quadros 11 e 12 exemplificam recursos que podem ser utilizados para o versionamento e produção de código. Contudo, caso as necessidades planejadas não forem supridas por esses artifícios é importante a busca por outras possibilidades. Dessa forma, ao fim da criação deste documento o ambiente de trabalho estará bem delimitado.

A seguir, o artefato de Diagrama de Componentes do Projeto deve apresentar uma representação visual dos principais módulos que compõem a linguagem, evidenciando suas funções e a forma como se comunicam durante o funcionamento do projeto. Esse diagrama tem como objetivo esclarecer as responsabilidades de cada componente e o fluxo de informações entre eles, facilitando a compreensão da organização interna da linguagem. Como apoio à elaboração desse artefato, recomenda-se a utilização da listagem de ferramentas produzida na fase de Reconhecimento (Quadro 06), uma vez que os componentes podem estar associados às tecnologias empregadas na implementação da linguagem.

Posteriormente, o diagrama de Estrutura de Diretórios deve retratar a divisão das pastas do projeto, definindo de forma clara que tipo de arquivos existirá e o que eles devem conter. Esse artefato é relevante para a manutenção do padrão de projeto tornando a solução tecnológica mais organizada e escalável.

O quarto artefato é o Organograma da Equipe, que define o grupo de trabalho e seus respectivos papéis dentro do processo de construção da linguagem. Este documento é apropriado para projetos de grande escala que contam com uma grande equipe. Em contrapartida, em projetos individuais ou de pequena dimensão, a produção deste artefato não se mostra necessária, uma vez que não há divisão de responsabilidades a ser formalizada.

O próximo artefato é o diagrama de fluxo de trabalho que define a metodologia para a adição de novas funcionalidades na linguagem. Esse diagrama pode contemplar desde a criação da tarefa em uma ferramenta de gestão até o encaminhamento para revisão por outro membro da equipe. A definição detalhada desse fluxo é essencial para restringir a possibilidade de fugas da organização da equipe.

Por fim, o documento de licenciamento e distribuição é recomendável para projetos comerciais de grande porte, pois nele são definidas questões jurídicas do projeto, incomuns para projetos pessoais.

Quadro 13 - Elementos que podem estar presentes no documento de licenciamento e distribuição

Elemento	Objetivo
Identificação do Projeto	Nome, versão, responsáveis e contexto de aplicação.
Modelo de Licenciamento	Define se ele vai ser código aberto, possuir licença comercial ou partes abertas e partes restritas.
Direitos e Restrições de Uso	Definir permissões de uso e modificação além de definir se é de uso comercial, sublicenciamento ou de patentes
Política de Distribuição	Definição se o projeto será disponibilizado publicamente em alguma plataforma e estratégia de distribuição comercial
Estratégia de manutenção e suporte	Listagem de estratégias para manter a linguagem após a produção de sua versão inicial.
Fonte de financiamento	Se o projeto foi patrocínio, venda, doações ou outro meio de fonte de financiamento.

Fonte: A autora.

O Quadro 13 sugere possíveis seções para o último artefato, mas elas devem advir das necessidades específicas do projeto. Os tópicos abrangem desde licenciamento e restrição até mesmo estratégia para manutenção da linguagem durante todo o período de uso.

5.3.2 Exemplo prático

Inicialmente, deve ser produzido na fase de arranjo o documento de ambiente de trabalho. Na tabela abaixo iremos demonstrar quais os elementos escolhidos para a composição de suporte do NiloScript.

Quadro 14 - Exemplo de documento de ambiente de trabalho no NiloScript

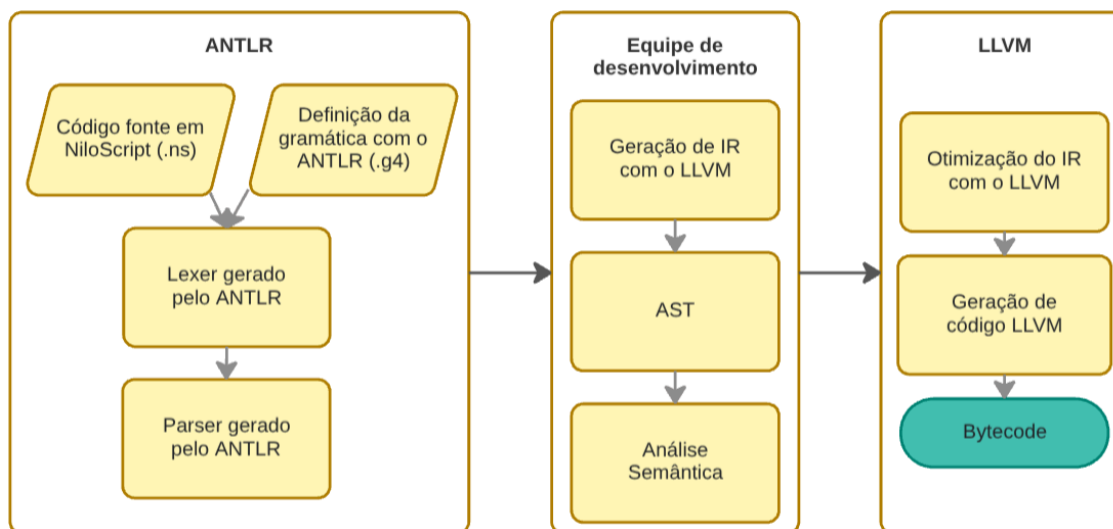
Utilidade	Ferramenta
Local de armazenamento e versionamento de código	Github
Ambiente de desenvolvimento integrado (IDE)	VsCode
Sistema operacional	Linux

Fonte: A autora.

O fato do NiloScript ser um projeto pequeno não houve uma listagem grande de componentes presentes no ambiente de trabalho. Dessa forma, a adição de novos componentes de ambiente é recomendada em caso que o projeto de linguagem necessite.

A seguir, será demonstrado o Diagrama de Componentes do Projeto produzido para a linguagem NiloScript.

Figura 22 - Exemplo de Diagrama de Componentes do Projeto do NiloScript

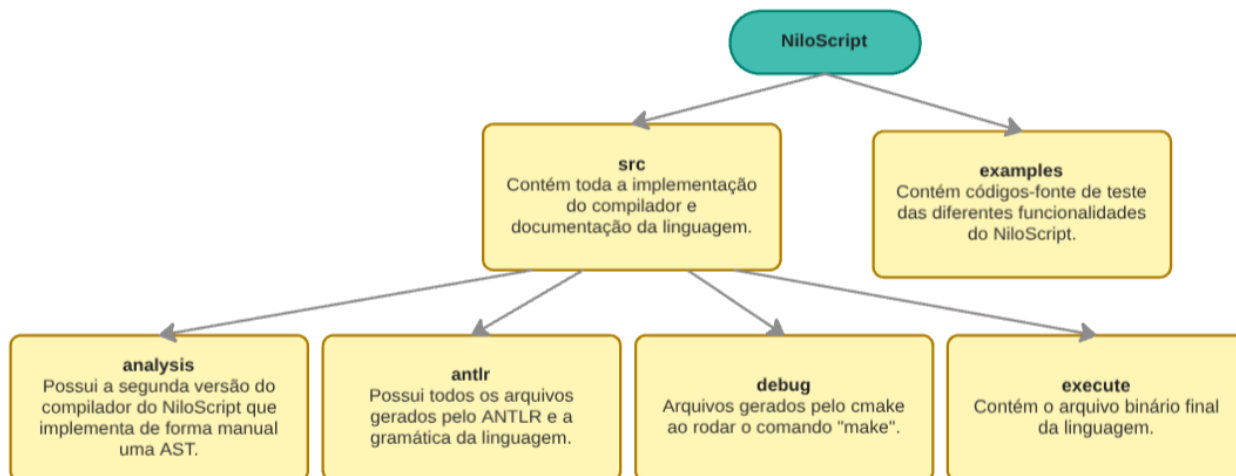


Fonte: A autora.

Ao final, o diagrama apresentado na Figura 22 deve representar de forma clara os principais componentes que compõem o projeto da linguagem, evidenciando suas responsabilidades e a forma como se comunicam para garantir seu funcionamento. Essa representação permite compreender a organização do projeto, identificando os módulos responsáveis pelas diferentes etapas do processo de compilação e o fluxo de informações entre eles.

Logo após, o diagrama de estrutura de diretórios deve se interligar mais com a concepção de implementação do código. Além disso, neste artefato, deve ser detalhado qual a funcionalidade de cada diretório e o que ele irá armazenar.

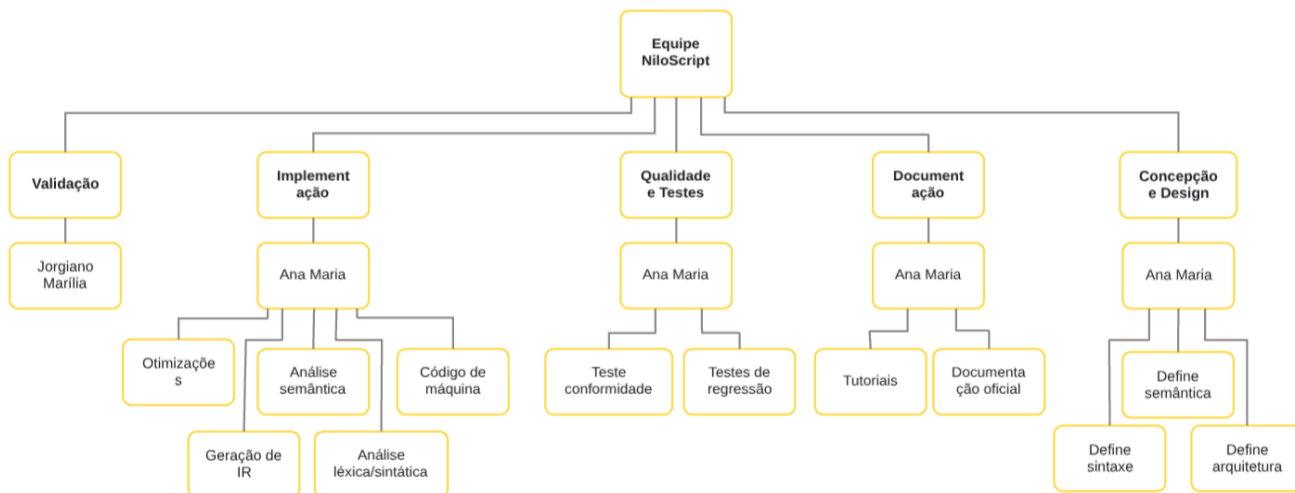
Figura 23 - Exemplo de diagrama de estruturas de diretórios do NiloScript



Fonte: A autora.

Posteriormente, apesar da linguagem NiloScript não possuir uma equipe de desenvolvimento extensa por surgir a partir de um projeto acadêmico individual, o organograma da equipe foi produzido para fins didáticos. A Figura abaixo representa uma estrutura sugerida para o artefato.

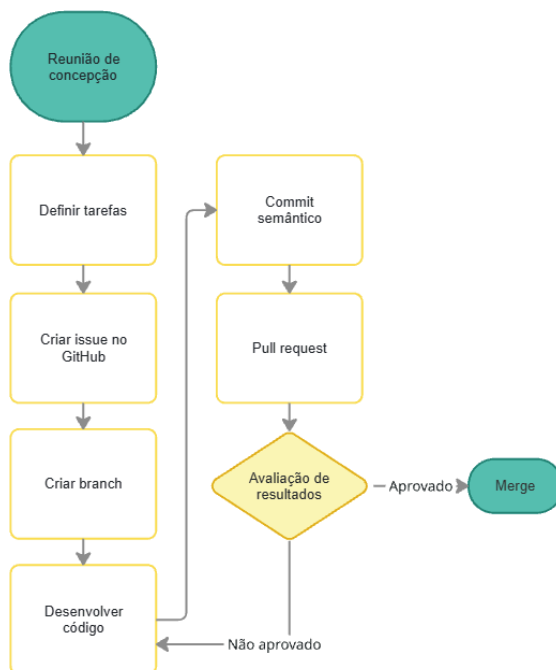
Figura 24 - Exemplo de organograma da equipe do NiloScript



Fonte: A autora.

De forma consecutiva, o diagrama de fluxo de trabalho deve definir objetivamente a sequência padrão de trabalho para o incremento de uma nova funcionalidade.

Figura 25 - Exemplo de diagrama de fluxo de trabalho do NiloScript



Fonte: A autora.

Por fim, o artefato produzido foi o documento de licenciamento e distribuição. O documento apresentado na Figura 26 foi criado para fins didáticos pois ele é recomendável para grandes projetos de linguagem comerciais, cenário que não se adequa a linguagem criada para este modelo de processo.

Figura 26 - Exemplo do documento de licenciamento e distribuição do NiloScript

Identificação do projeto**Nome:** NiloScript**Versão:** 1.0**Responsáveis:** Ana Maria**Contexto de aplicação:** Linguagem simplificada voltada para ensino de compiladores e geração de código intermediário.**Modelo de Licenciamento**

Código aberto, mas com restrições: não é permitido modificar diretamente o código-fonte original. É possível realizar **forks** e criar projetos derivados, desde que respeitando os termos de uso.

Direitos e Restrições de Uso

- Uso permitido para fins acadêmicos e não comerciais.
- É possível distribuir forks, mas não modificar o repositório principal. Mas esses forks não podem ser comercializados.

Política de Distribuição

O projeto será disponibilizado publicamente no GitHub. Não há estratégia de distribuição comercial.

Estratégia de Manutenção e Suporte

- Correções pontuais de erros identificados.
- Atualizações ocasionais para compatibilidade com novas versões de ferramentas externas (ex.: LLVM, ANTLR).
- Suporte comunitário via issues e discussões eventuais no github.

Fonte de Financiamento

O projeto **não possui financiamento**. É mantido de forma independente, sem patrocínio, venda ou doações.

Fonte: A autora.

5.4 CODIFICAÇÃO

A fase de codificação é a mais flexível do NILO, nesse momento será desenvolvido a versão inicial do código seguindo o fluxo de trabalho definido na etapa de estruturação. Sua flexibilidade decorre da dependência direta com as ferramentas escolhidas para o projeto, ou pela ausência delas em casos de codificação manual.

Figura 27 - Artefatos que devem ser produzidos na fase de codificação



Fonte: A autora.

Assim como indicado pela Figura 27 essa fase corresponde à materialização do projetado nos artefatos das etapas anteriores. Contudo, o presente trabalho irá representar o desenvolvimento das funcionalidades de uma linguagem através da produção de um compilador, para tradução de um código-fonte para um código de

máquina. A forma que a linguagem será traduzida consiste em uma decisão projetual da equipe.

5.4.1 Mão na massa

A primeira tarefa desta fase é a preparação do ambiente. Isso significa que é necessário a configuração de todas as máquinas dedicadas ao projeto para o suporte devido às ferramentas escolhidas nas etapas anteriores para o desenvolvimento da linguagem. Esse processo envolve a criação, instalação e configuração de todas as tecnologias definidas no documento de ambiente de trabalho. O fluxo estabelecido deve ser seguido para todos os desenvolvedores envolvidos no projeto.

É válido ressaltar que o fluxo padronizado de produção de código, definido no diagrama de fluxo de trabalho, é essencial durante todas as tarefas da etapa de codificação, até mesmo na fase inicial de configuração.

Por fim, o último passo é o desenvolvimento das funcionalidades da linguagem. É fundamental que esse processo seja conduzido em concordância com os artefatos produzidos nas fases anteriores, para a garantia de conformidade com os requisitos e estruturas definidas para a linguagem.

5.4.2 Exemplo prático

Neste tópico será explorado, em mais detalhes, como foi realizada, passo a passo, a implementação do compilador da linguagem NiloScript. Como apresentado na fase de Definição, a linguagem já possui sua especificação sintática formalizada por meio da gramática, a qual servirá como base para todas as etapas de implementação do compilador descritas nesta seção.

Para fins didáticos, foram criadas duas versões do compilador do NiloScript. Nesta seção será apresentada a versão mais simplificada e recomendada para desenvolvedores que estejam implementando sua primeira linguagem de programação utilizando as mesmas tecnologias empregadas neste trabalho. Nesse fluxo simplificado, a geração da representação intermediária ocorre em conjunto com a análise semântica a partir da árvore sintática concreta, dispensando a construção de uma árvore sintática abstrata.

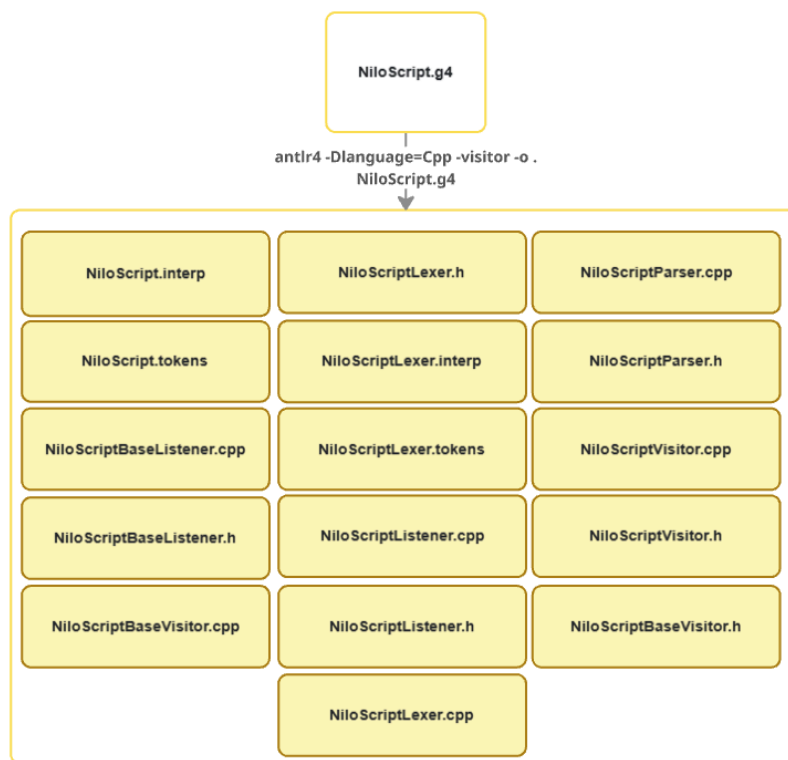
Ressalta-se que a implementação apresentada não representa um processo determinístico, mas sim uma demonstração de como o compilador do NiloScript foi desenvolvido. Grande parte das decisões adotadas decorreu das ferramentas escolhidas para o projeto. Dessa forma, o fluxo descrito deve ser compreendido como

uma das possíveis estratégias de implementação, e não como uma regra para o desenvolvimento de compiladores.

5.4.2.1 *Lexer e Parser*

Com a especificação da gramática definida na fase de Definição, o primeiro passo da implementação do compilador consiste em utilizá-la para gerar automaticamente o analisador léxico (*lexer*) e o analisador sintático (*parser*).

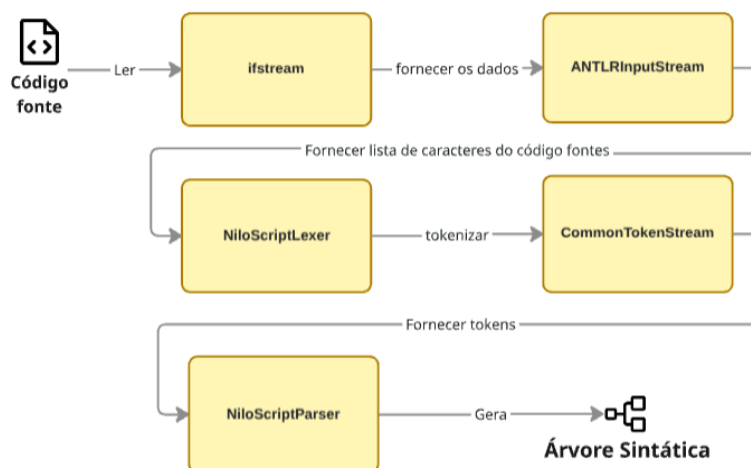
Figura 28 - Comando utilizado para gerar *lexer* e *parser* a partir da gramática.



Fonte: A autora.

Para isso foi-se utilizado um comando no terminal, representado na Figura 28, para que a ferramenta ANTLR gerasse de forma automatizada e com base na gramática, arquivos que correspondem aos analisadores léxico e sintático da linguagem, além de utilitários como os tokens. Esses arquivos possibilitam a geração de uma árvore sintática correspondente para qualquer código-fonte fornecido que siga as regras definidas na gramática.

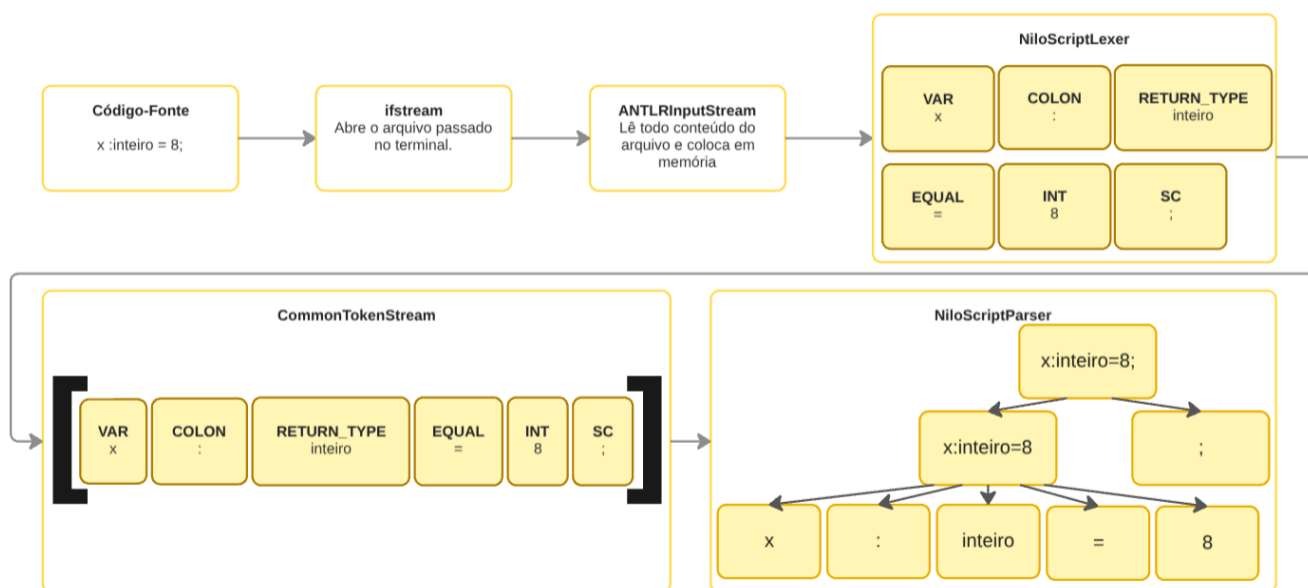
Figura 29 - Comandos utilizados para a geração de uma árvore sintática a partir de um código-fonte.



Fonte: A autora.

Em seguida, ocorre uma sequência de chamadas aos arquivos gerados pelo ANTLR, assim como representado na Figura 29, para que o lexer e o parser consigam transformar o texto-fonte de uma linguagem em uma árvore sintática. Primeiramente, por meio da classe **ifstream** do C++ é realizada a leitura do texto-fonte fornecido pelo programador para que a classe **ANTLRInputStream** possa ler esse arquivo e transformar todo o conteúdo em um *buffer* (armazenamento intermediário) de caracteres. Em seguida, essa sequência de caracteres gerada é fornecida para o **NiloScriptLexer**, analisador léxico da linguagem NiloScript, para que ao fim ele resulte em *tokens* que representam todo o código fornecido. Porém, o fornecimento para o parser dos *tokens* gerados não pode ocorrer de forma direta necessitando da classe intermediária **CommonTokenStream** que serve para armazenar em listas todos os *tokens* gerados pelo *lexer* permitindo que o parser consiga olhar o próximo *token*, sem consumi-lo, além de possibilitar o acesso de todos os tokens por meio de índices. Por fim, a lista de tokens gerados e tratados são fornecidas para o *parser* da linguagem o **NiloScriptParser**, que é responsável pela criação de uma árvore sintática que ordena todos os tokens fornecidos. A implementação desse fluxo na linguagem NiloScript está presente no repositório Github da linguagem: <https://github.com/namariaa/Nilo/blob/main/NiloScript/src/Main.cpp>.

Figura 30 - Exemplo do fluxo completo de comandos partindo de um código-fonte



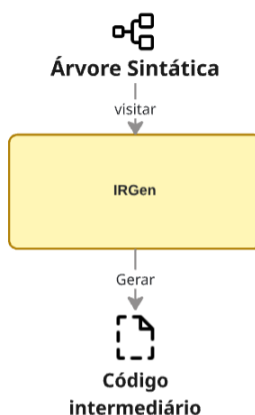
Fonte: A autora.

A Figura 30 demonstra um exemplo prático do fluxo completo da utilização dos arquivos de lexer, parser e utilitários na geração de uma árvore sintática. Partindo da abertura e leitura do código-fonte “x :inteiro = 8;” pelas classes **ifstream** e **ANTLRInputStream** respectivamente o **NiloScriptLexer** transforma todo o programa nos tokens “VAR” para a variável “x”, “COLON” para o símbolo “:”, “RETURN_TYPE” para o tipo “Inteiro”, “EQUAL” para o símbolo de “=”, “INT” para o número inteiro 8 e “SC” para o símbolo de “;”. A seguir, a classe **CommonTokenStream** organizou todos os tokens fornecidos pelo *lexer* em uma lista, para que por fim o **NiloScriptParser** ordenasse as informações em uma estrutura de dados em formato de árvore (sintática).

5.4.2.2 Geração de código

Para finalizar a implementação do compilador do NiloScript que ocorreu de forma manual, prosseguiremos com o entendimento sobre a geração de código intermediário.

Figura 31 - Fluxo de geração de uma IR partindo da árvore sintática



Fonte: A autora.

No NiloScript, por meio da classe **IRGen** a árvore sintática construída a partir do código-fonte, conforme descrito na seção 5.4.2.2, é utilizada como estrutura de dados de suporte para a geração de código intermediário, assim como demonstrado na Figura 31. Essa árvore é percorrida e transformada em uma representação compatível com o LLVM IR (*intermediate representation*). O LLVM, por sua vez, interpreta essas instruções em nível intermediário e realiza otimizações antes de traduzi-las para código de máquina.

Figura 32 - Exemplo de comando para a geração de uma representação intermediária de um número inteiro.

```

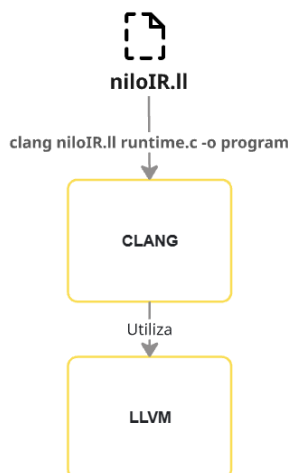
llvm::Value* inteiro =
llvm::ConstantInt::get(llvm::Type::getInt32Ty(*Container),
std::stoi(context->INT()->getText()));
  
```

Fonte: A autora.

Cada linha ou bloco de código do texto-fonte é transformado em instruções geradas com o apoio da ferramenta LLVM. Essa representação é compreensível pelo próprio LLVM e funciona como uma camada intermediária entre a linguagem de alto nível e o código de máquina. Cada instrução representa uma operação de baixo nível, como alocação de variáveis (*alloca*), carregamento e armazenamento em memória (*load*, *store*), operações aritméticas (*add*, *sub*, *mul*, *div*), controle de fluxo (*br*, *phi*) e chamadas de função (*call*). Essas instruções são organizadas em um **módulo**, que reúne toda a representação intermediária do código-fonte. A Figura 32 ilustra um exemplo de representação de um número inteiro por meio da ferramenta, evidenciando

apenas uma entre as diversas possibilidades oferecidas por sua infraestrutura. Ao final do processo, toda a representação intermediária derivada do código-fonte é transformada em um arquivo com extensão “.ll”, que contém as instruções na forma de LLVM IR. Esse arquivo corresponde ao módulo citado anteriormente e reúne todas as operações de baixo nível compreensíveis pelo próprio backend do LLVM.

Figura 33 - Execução do código intermediário



Fonte: A autora.

Por último, conforme explicado na Seção 4.3 e exemplificado na Figura 33, utilizou-se o Clang como intermediário na geração e execução de código de máquina, fazendo uso do backend do LLVM. Dessa forma, o compilador do NiloScript foi finalizado com suporte a todas as funcionalidades previstas na gramática.

5.5 TESTE

Os testes de sistemas são essenciais para revelar defeitos nas linhas de código. Na produção de linguagens a etapa de testes é necessária durante todo o desenvolvimento, para a validação e garantia da confiabilidade do código gerado. Essa confiabilidade é assegurada pela verificação da compatibilidade de código com os requisitos planejados nas etapas anteriores.

Nesse contexto, existem diversos tipos diferentes de testes, que avaliam níveis distintos da linguagem. Para uma maior adequação a essa variabilidade de tipos, foram criadas diversas formas de produzir avaliações de código, sendo necessária a utilização e implementação de acordo com as necessidades surgidas durante o desenvolvimento.

Quadro 16 - Alguns tipos de testes que podem ser realizados em uma linguagem

Nome	Descrição
Teste de Parsing	Avalia se a gramática aceita/rejeita entradas corretamente.
Teste de Geração de IR	Examina o formato do LLVM IR.
Teste de Fuzzing	Avalia respostas da linguagem a entradas aleatórias e erradas.

Fonte: A autora.

Além disso, atualmente, existem diversas ferramentas que auxiliam na realização dos diferentes tipos de testes demonstrados no Quadro 16. Porém, a existência de arcabouços que ajudem nessa etapa não revoga a possibilidade de realização das avaliações de forma manual. A seguir, iremos analisar diferentes ferramentas que podem ser utilizadas nessa execução:

Quadro 17 - Ferramentas que podem ser utilizadas

Ferramenta	Uso	Tipo de teste que ele faz
LLVM Lit	Ferramenta integrada ao LLVM para execução de testes.	Testes funcionais e de regressão em compiladores e ferramentas baseadas em LLVM.
ANTLR TestRig (grun)	Utilitário incluído no runtime do ANTLR que permite testar gramáticas. Mostra como a entrada é analisada pelas regras definidas.	Testes de parsing e validação de gramática.
ANTLR4TestGenerator	Ferramenta que gera automaticamente casos de teste a partir de gramáticas ANTLR4.	Testes automatizados de gramática, cobrindo diferentes caminhos de análise.
Hypothesis	Gera entradas automaticamente para verificar propriedades	Testes explorando casos extremos e entradas inesperadas para validar

	definidas pelo desenvolvedor.	robustez da linguagem.
QuickCheck	Similar ao Hypothesis, mas aplicada em ambientes funcionais.	Testes explorando casos extremos e entradas inesperadas para validar robustez da linguagem.

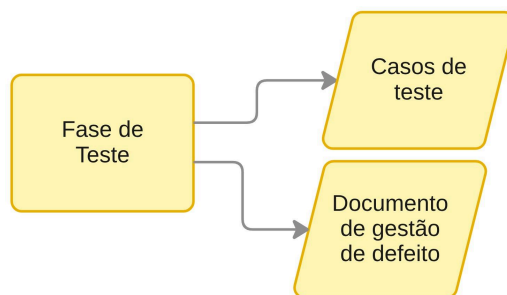
Fonte: A autora.

A partir da análise das tecnologias de suporte à realização de testes exemplificadas no Quadro 17, iremos partir para a realização da atividade prática.

5.5.1 Mão na massa

O dever a ser cumprido nesta fase será sistematicamente dividido em duas atividades fundamentais (Figura 34). O objetivo central consiste em validar se a linguagem atende aos requisitos e funcionalidades especificados durante seu desenvolvimento, bem como registrar e acompanhar os defeitos identificados durante sua execução.

Figura 34 - Artefatos que devem ser produzidos na fase de teste



Fonte: A autora.

Inicialmente, devem ser produzidos os Casos de Teste da linguagem. Esses casos devem ser elaborados com base nos requisitos e nas funcionalidades definidas nas fases anteriores, orientando a validação do comportamento esperado da linguagem. Os casos de teste podem ser representados por programas escritos na própria linguagem, executados manualmente, ou automatizados por meio de ferramentas específicas. Independentemente da abordagem adotada, recomenda-se a elaboração de casos que contemplem tanto cenários de sucesso quanto cenários de exceção, permitindo verificar não apenas a correta execução das funcionalidades

implementadas, mas também a capacidade da linguagem de identificar e tratar entradas inválidas e demais situações excepcionais.

Após a elaboração dos casos de teste, deve ser realizada sua execução. Essa etapa pode ocorrer manualmente ou com o auxílio de ferramentas de testes automatizados, conforme as características do projeto e das tecnologias empregadas. Durante a execução, os resultados obtidos devem ser comparados com os resultados esperados definidos nos casos de teste, permitindo validar se o comportamento da linguagem está de acordo com sua especificação.

Sempre que um comportamento inesperado for identificado durante a execução dos testes, torna-se necessária sua documentação por meio do Documento de Gestão de Defeito. Esse artefato possui o propósito de registrar, acompanhar e controlar os problemas encontrados ao longo do desenvolvimento da linguagem, evitando que defeitos sejam esquecidos ou reincidam em versões futuras.

Há diversas formas de documentar e acompanhar a evolução dos defeitos durante o desenvolvimento de uma linguagem. O Quadro 18 apresenta alguns exemplos de artefatos que podem ser utilizados como apoio à gestão desses problemas.

Quadro 18 - Diferentes formas de gerir e documentar defeitos

Artefato de gestão de defeito	Descrição
Diagrama de Tendência de Defeitos	Representação gráfica da evolução do número de defeitos ao longo do tempo. Permite o monitoramento da linguagem durante o processo de desenvolvimento.
Relatório de Densidade de Defeitos	Documento que relaciona a quantidade de defeitos encontrados ao tamanho do código ou ao número de funcionalidades implementadas. É utilizado para medir a qualidade e identificar partes mais suscetíveis a falhas.
Relatórios de tempo de permanência	Registro de intervalo de tempo entre a identificação de um defeito e sua resolução.

Fonte: A autora.

Para relatar esses defeitos podem ser utilizados os artefatos demonstrados no Quadro 18. Para produzir a documentação do desvio de funcionalidade de linhas de código, empregamos parâmetros para a classificação dos erros encontrados durante os testes, podendo ser pelo **status**, **local**, **prioridade** e **gravidade**.

Quadro 19 - Parâmetros de classificação de *bugs*

Parâmetro	Descrição
Status	Classifica o bug em “pendente” e “resolvido” mas pode ser flexível e receber outras denominações e variações.
Local	Origem do erro, onde foi encontrado o comportamento inesperado.
Prioridade	Prioridade de resolução daquele problema. Pode ser classificado em “Alta”, “Média” e “Baixa”.
Gravidade	O quão impactante pode ser aquele problema para a linguagem “Crítico”, “Moderado” e “Pequeno”.

Fonte: A autora.

Na última atividade desta fase, deve ser produzido o Documento de Gestão de Defeito, registrando os problemas encontrados durante a execução dos casos de teste e classificando-os conforme os parâmetros apresentados no Quadro 19. A manutenção contínua desse documento, aliada à atualização periódica dos casos de teste, contribui para aumentar a confiabilidade da linguagem e facilitar sua evolução ao longo do desenvolvimento.

5.5.2 Exemplo Prático

Durante o desenvolvimento do NiloScript, foi produzido inicialmente um conjunto de Casos de Teste destinado a validar as funcionalidades implementadas na linguagem. Como estratégia de validação, optou-se pela execução manual dos testes, permitindo acompanhar detalhadamente o comportamento do compilador em cada etapa do processo.

Figura 35 - Caso de teste do NiloScript

Caso de teste: Declaração e chamada de função

O principal objetivo é validar a declaração de funções, a passagem de parâmetros, a execução de chamadas de função, o retorno de valores e a utilização de funções nativas de entrada e saída da linguagem, verificando se o compilador interpreta corretamente.

Código de teste

```

funcionalidade soma (x :inteiro, y :inteiro) :inteiro {
  resultado :inteiro = x + y;
  retorne resultado;
}

a :inteiro = pegaInteiro;
b :inteiro = pegaInteiro;
c :inteiro = soma(a, b);
mostrarInteiro(c);
  
```

Passo a Passo para a realização do teste

1. Compilar o programa.
2. Verificar a geração do código intermediário.
3. Executar o programa.
4. Comparar a saída obtida com o resultado esperado.

Resultado esperado

Considerando as entradas: **10 e 20**
 O programa deve ser compilado sem erros e produzir a seguinte saída: **30**

Critério de aprovação

O teste será considerado Aprovado caso:

- o programa seja compilado sem erros;
- a geração do código intermediário seja concluída corretamente;
- a função soma receba os parâmetros corretamente;
- a instrução retorne devolva o valor esperado;
- as funções nativas mostrarCaracteres, pegaInteiro e mostrarInteiro sejam executadas corretamente;
- a saída do programa corresponda ao resultado esperado.

Resultado obtido

Aprovado.

Fonte: A autora.

Os casos de teste foram elaborados contemplando tanto cenários de sucesso quanto cenários de erro. Para isso, foram produzidos códigos-fonte em NiloScript que exercitavam as diferentes funcionalidades da linguagem, especificando para cada caso o comportamento esperado. Dessa forma, foi possível verificar se o compilador produzia os resultados corretos para entradas válidas e se identificava adequadamente situações de erro decorrentes do descumprimento das regras léxicas, sintáticas ou semânticas da linguagem.

À medida que novas funcionalidades eram implementadas, os casos de teste correspondentes eram executados. Quando o comportamento observado diferia do resultado esperado, o defeito era registrado no Documento de Gestão de Defeito e a implementação retornava à fase de Codificação para realização das correções necessárias. Após a resolução do problema, o caso de teste era executado novamente para confirmar que o defeito havia sido eliminado sem comprometer as demais funcionalidades da linguagem.

Embora o NiloScript tenha adotado predominantemente testes manuais, recomenda-se, sempre que possível, a utilização de ferramentas de testes

automatizados como complemento ao processo de validação. Entretanto, a escolha da estratégia de testes deve considerar as características da linguagem desenvolvida e a disponibilidade de ferramentas compatíveis, de modo que a abordagem adotada efetivamente contribua para a qualidade do compilador.

6 CONCLUSÃO

Embora a motivação do presente modelo de processo se justifique na tentativa de suprir o desprovimento de ensino de fundamentos de linguagens de codificação no curso de TADS, o modelo de processo busca alcançar um amplo público com diferentes níveis de experiência. Além disso, o NILO procura, por meio de sugestões de melhorias dos seus utilizadores, alcançar uma gama de conceitos e público maior com o objetivo de colaborar na disseminação do conhecimento de linguagens de programação.

REFERÊNCIAS

NEVES, Maria Helena de Moura. **A gramática: História, teoria e análise, ensino**. São Paulo: Editora Unesp, 2002. 288 p.

PROJECT, Llvm. **Clang: a C language family frontend for LLVM**. 2026. Disponível em: <https://clang.llvm.org/>. Acesso em: 02 abr. 2026.

CRUZ, Heron Sampaio da et al. **Compilação Just-in-Time**. 2008. Disponível em: <http://pesquompile.wikidot.com/compilacaojustintime>. Acesso em: 12 abr. 2026.

AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. São Paulo: Pearson Addison Wesley, 2007.

SEBESTA, Robert W.. **Conceitos de linguagens de programação**. Porto Alegre: Bookman, 2011. 758 p.

ARIMOTO, Maurício; OLIVEIRA, Weldrey. Dificuldades no Processo de Aprendizagem de Programação de Computadores: um survey com estudantes de cursos da área de computação. **Anais do Workshop Sobre Educação em Computação (Wei)**, [S.L.], p. 244-254, 12 jul. 2019. Sociedade Brasileira de Computação - SBC. <http://dx.doi.org/10.5753/wei.2019.6633>.

LEE, Kent D.. **Foundations of Programming Language**. Cham: Springer, 2017.

JOHANN, Marcelo. **Gramáticas Livres de Contexto**. [S. L.], 2020. 30 slides, color. Disponível em: <https://www.inf.ufrgs.br/~johann/comp/aula04.glc.pdf>. Acesso em: 02 abr. 2026.

FREECODECAMP. **Interpreted vs Compiled Programming Languages: What's the Difference?** 2020. Disponível em: <https://www.freecodecamp.org/news/compiled-versus-interpreted-languages/>. Acesso em: 02 mar. 2026.

CHOMSKY, Noam. On certain formal properties of grammars. **Information And Control**, [S.L.], v. 2, n. 2, p. 137-167, jun. 1959. Elsevier BV. [http://dx.doi.org/10.1016/s0019-9958\(59\)90362-6](http://dx.doi.org/10.1016/s0019-9958(59)90362-6).

LIMA, Bruno. **Otimização**. 2022. Disponível em: <https://wiki.inf.ufpr.br/computacao/doku.php?id=o:otimizacao>. Acesso em: 02 abr. 2026.

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO RIO GRANDE DO NORTE. **Projeto Pedagógico do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas na modalidade presencial**. Natal, 2012. Disponível em: https://diatinf.ifrn.edu.br/wp-content/uploads/2024/03/PPC__Tecnologia_em_Analise_e_Developolvimento_de_Sistemas_2012.pdf. Acesso em: 09 set. 2025.

RUBY. **Sobre o Ruby**. Disponível em: <https://www.ruby-lang.org/pt/about>. Acesso em: 02 abr. 2026.

PARR, Terence. **The Definitive ANTLR 4 Reference**. S. L: Pragmatic Bookshelf, 2013. 328 p.

MIGHT, Matthew. **The language of languages**. Disponível em: <https://wiki.inf.ufpr.br/computacao/doku.php?id=c:compiladores>. Acesso em: 02 abr. 2026.

LLVM Project (org.). **The LLVM Compiler Infrastructure**. 2026. Disponível em: <https://llvm.org/>. Acesso em: 4 abr. 2025.

PYTHON. **The Python Tutorial.** Disponível em:
<https://docs.python.org/3/tutorial/index.html#tutorial-index>. Acesso em: 02 abr. 2026.

VARCK. **Vark.** Disponível em: <https://vark-learn.com/#>. Acesso em: 10 abr. 2026.